ShiftSwap: A Shift Schedule and Substitute Management System for Small Part-Time Workplaces

Requirements specification and analysis models

Table 1. Team 7

Name	Student ID	Affiliation	Role
Hyewon Kim (김혜원)	24102028	Dept. of Industrial Engineering, ITM Program, SeoulTech	Requirements lead
Hyoungdo Kim (김형도)	18102075	Dept. of Industrial Engineering, ITM Program, SeoulTech	Project manager and design/implementation lead
Mingyeong Kim (김민경)	23102004	Dept. of Industrial Engineering, ITM Program, SeoulTech	UI and quality lead

Course: Software Engineering, Seoul National University of Science and Technology (SeoulTech)
Submitted to: jihjim@gmail.com
Date: 16 October 2026

Contents

1. Introduction
    1.1 Response to the proposal feedback
    1.2 Changes since the proposal
2. Problem description and system environment
    2.1 Background and current process
    2.2 Problems P1–P3
    2.3 Proposed solution
    2.4 System environment
    2.5 Demo store
    2.6 Actors
3. Requirements elicitation
4. Functional requirements specification
    4.1 Functional requirements
    4.2 Dependencies of increment 1
    4.3 Business rules
    4.4 Use-case descriptions
5. Non-functional requirements specification
6. Scenarios
7. UML diagrams for requirements analysis
    7.1 Use case diagram
    7.2 Activity diagrams
    7.3 State machine diagrams
    7.4 Analysis class diagrams
    7.5 Analysis sequence diagram
8. Main user interface screens
9. Requirements validation
10. Progress, risks and next steps
Appendix A. Test cases
Appendix B. Pages and role-based access
References

1. Introduction

This interim report specifies the requirements of ShiftSwap and presents the UML models used to analyse them. ShiftSwap is a web application for a small store with part-time workers. It manages the weekly shift schedule, finds substitutes when a worker cannot work a shift, and computes weekly holiday allowance and monthly pay under Korean labour law.

The proposal (Week 5) defined the problem, the goals, the nine main functions and the project plan. This report refines them: problem and system environment (Section 2), elicitation (3), functional and non-functional requirements (4, 5), scenarios (6), analysis models (7), main screens (8), validation (9) and progress (10). Appendix A lists the test cases, which are fixed before implementation, and Appendix B the pages and the role-based access. The team follows the incremental model with prototyping of the user interface (Chapter 1) [6, 7]: increment 1 delivers the *Must* use cases and increment 2 the *Should* use cases. All identifiers (UC, FR, NFR, BR, TC) come from one project specification that the team uses as the single source of truth; the same identifiers appear in the models, the database functions and the tests. Identifiers are never renumbered: a requirement that was removed keeps its number unused.

1.1 Response to the proposal feedback

The instructor's feedback on the proposal rated the feasibility as high, the risk from external APIs as low and the need for commercial packages as none, and asked for three priority improvements: the dependencies of increment 1 on the *Should* use cases, server-side atomicity and the expiry method, and a stronger differentiator together with earlier test cases. Table 2 lists each point of the feedback, the change made in response and where it is described.

Table 2. Response to the proposal feedback

Feedback	Change	Where
“First acceptance wins” is not atomic when a static site calls the database in several steps; it must be one database function (RPC).	Every user action is one call of a database function, run as one transaction. Acceptance is one conditional `UPDATE … WHERE status = 'REQUESTED'` inside `accept_sub_request`; a concurrent second acceptance changes no row and is refused.	2.4, BR-02, NFR-04, S2, Figure 11
The executor of expiry (UC-11, System Clock) is not determined (pg_cron or evaluation at query time).	Evaluation at query time: the database function `run_system_clock()` runs whenever a screen is loaded, and the functions that answer, decide or cancel a request check the deadline again inside their own transaction.	2.6 (A3), FR-18, BR-12, UC-11, S4
The vibe-coding tool is not determined.	The system is developed and verified with Claude Code. The final prompt is written for Lovable, the tool with which the instructor re-runs it; a first trial in Lovable is planned for Week 9 and a full re-run in Week 13.	2.4.3, NFR-08, Section 10

Feedback	Change	Where
Notifications are in-app only, so a request is seen only when the app is opened; this limits the quick answers of the proposal's goals G1 and G2 and should be stated as a risk.	New risk R7 with mitigations: unread count in the header, the reply deadline on every request card, screens refreshed every 10 seconds and when the window regains focus.	Section 10 (Table 45), NFR-15
The dual implementation with Supabase and sql.js should be limited to a fallback for demonstrations.	The browser keeps no copy of the data and there is no second data mode: one PostgreSQL database holds the data and the rules. The scheduled keep-alive read, which the feedback found adequate, stays.	2.4, NFR-05, NFR-08, NFR-16
UC-08 (Must) needs “every shift WORKED” (BR-06), but attendance (UC-07) and settings (UC-10) are *Should*: increment 1 alone cannot decide eligibility.	BR-06 now counts a past shift as worked unless the owner marked it ABSENT. Marking a no-show absent is FR-25 of UC-03 (*Must*). The settings have seeded defaults. UC-14, which reuses the payroll of UC-09, moves to *Should*.	4.2 (Table 9), BR-06, FR-25, UC-03
Under the default policy EXCUSED a swap never changes either worker's eligibility, so the differentiator “eligibility change at approval” always shows “no change”; substantive information such as a premium estimate is needed.	The approval preview now shows each worker's hours, holiday allowance and pay change for the week, the premium change in a workplace with five or more regular employees, and the change in the store's labour cost (BR-16).	2.2, FR-11, BR-16, S3, S7, S8
Night hours 22:00–06:00 (BR-08), fixed shifts given as weekday, start and end, and the week ending Sunday 24:00 do not fit together: a shift past midnight cannot be expressed.	A time-line rule: an end time not later than the start time means the next day; the shift belongs to the date and week on which it starts; night hours and overlaps are computed on the time line; clock times are checked for plausibility.	FR-02, BR-05, BR-08, BR-17, S9
UC-13 (role choice without password) and NFR-11 (others' wages not shown) are only screen-level on a public URL; state that only fictional data are used.	The sign-in screen and the requirements state that all data are fictional demo data and that privacy is screen-level only.	FR-20, NFR-11, UC-13

Feedback	Change	Where
Move the test cases before Week 9.	The test cases with their boundary values are listed in Appendix A and fixed in Week 8; the database functions are already tested against them.	9.3, Appendix A, Section 10

1.2 Changes since the proposal

Besides the response to the feedback, the team's internal review of the prototype changed three requirements of the proposal. Table 3 lists every change with its reason.

Table 3. Changes since the proposal

Change	Reason	Identifiers
Availability is no longer collected (use case UC-02 Register availability and the availability condition of the candidate rule are removed); every co-worker without an overlapping shift is asked.	In the team's internal review of the prototype, a request whose shift matched nobody's availability only told the owner that no candidate existed, which left the owner with nothing to act on; workers also had to keep their availability up to date.	UC-02, FR-03 (removed), BR-01
A request that nobody can take ends as FAILED; candidates can answer “can't”.	Follows from the change above: the requester learns at once to contact the owner, and the owner acknowledges the case.	FR-08, FR-22, BR-13
Past payroll drafts are prepared automatically (*Should*).	Found in the team's internal review of the payroll prototype: the owner should start from a draft on the first day of the month.	UC-14, FR-23, BR-14
UC-03 is renamed “Manage weekly schedule” and includes marking a no-show absent.	Feedback on increment 1 (Section 1.1).	UC-03, FR-25
The approval preview shows money; overnight shifts and plausible clock times are specified.	Feedback (Section 1.1).	FR-11, BR-05, BR-08, BR-16, BR-17
Rules are database functions; the browser keeps no data copy; the tools are named.	Feedback (Section 1.1).	NFR-04, NFR-05, NFR-08, NFR-14

2. Problem description and system environment

2.1 Background and current process

In small stores (cafés, convenience stores, restaurants) with fewer than ten part-time workers, each worker has fixed weekly shifts, for example every Monday 18:00–23:00. When a worker cannot come, the owner phones or messages other workers one by one to find a substitute. The owner then edits a spreadsheet by hand. At month end the owner must re-check each worker's weekly hours to decide whether the weekly holiday allowance (주휴수당) is owed.

Our reference store is the workplace of team member Hyewon Kim. The owner also acts as the manager. There, finding a substitute typically takes 3 to 5 contacts per request. The current process has four manual steps: (1) the worker informs the owner; (2) the owner contacts other workers one at a time; (3) the owner edits the spreadsheet and informs both workers; (4) at month end the owner re-counts hours and computes pay. None of these steps is recorded except the final spreadsheet.

Korean law adds rules that depend on these hours. A worker whose weekly contractual hours are 15 or more and who worked every scheduled day of the week is entitled to a paid weekly holiday (Labor Standards Act Art. 55 ①, Art. 18 ③, Enforcement Decree Art. 30 ①) [3, 4]. The 2026 minimum hourly wage is KRW 10,320 [1, 2]. Overtime and night premiums apply only in workplaces with five or more regular employees (Art. 11, 56) [3]. In such workplaces a part-time worker may work beyond the contractual hours only with the worker's consent and for at most 12 hours a week, and these hours earn a premium of at least 50 % (Fixed-term and Part-time Employees Act Art. 6) [20]; accepting a substitute shift usually creates such hours. The minimum wage may be reduced to 90 % during probation only if several conditions hold (Minimum Wage Act Art. 5 ②, Enforcement Decree Art. 3) [5].

2.2 Problems P1–P3

P1 Finding a substitute costs the owner repeated one-to-one contacts (reference store: typically 3–5 contacts per request).

P2 The schedule, weekly hours, pay and holiday-allowance eligibility of *two* workers change with every swap, and all recalculation is manual.

P3 Legal pay rules (minimum wage, holiday allowance, 5-employee threshold for premiums, overtime of part-time workers beyond contractual hours, probation reduction) are applied from memory, so errors are not caught before payday.

Scheduling products abroad already support a substitute workflow between co-workers. In Deputy a worker can offer a shift to co-workers who are qualified and available, and swaps can be set to require the manager's approval [10]. When I Work lets a worker offer a shift to all or selected eligible co-workers, subject by default to a manager's approval, and with its Find Replacement function a manager offers a shift to eligible workers, the first of whom to pick it up takes it [11, 12]. In Homebase a worker asks all or selected co-workers to cover a shift, and a manager gives final approval after one of them accepts [13]. Deputy's employee app is available in Korean, but managers who build schedules on its web interface must work

in English [14], and in the help centres of all three products we found no mention of the Korean weekly holiday allowance.

Korean products follow a different workflow, in which the manager edits the schedule or an employee sends a change request to the manager. Shiftee lets an employee request a new or changed shift and computes the weekly holiday allowance automatically [15, 16]; its help centre states that the allowance is recognised only when every contractual working day is worked [16]. Shopl lets employees request schedule changes that the manager approves or rejects [17]. Albami and Gig let the owner manage schedules and the holiday allowance [18, 19]. The public feature descriptions of these four products do not include a substitute request between co-workers. We found no product that combines the co-worker substitute request with the Korean pay rules at the moment of approval, that is, one that shows the owner before the decision what a swap does to both workers' hours, holiday allowance and pay, to the store's labour cost and, in a workplace with five or more regular employees, to premiums and the 12-hour limit for part-time workers.

2.3 Proposed solution

ShiftSwap replaces steps (2) to (4). A worker creates a substitute request for one of their shifts. The system asks every co-worker who could work at that time, in one step, through the in-app inbox. The first colleague to accept becomes the acceptor; if every colleague answers “can't”, the requester is told at once to contact the owner. Before deciding, the owner sees what the swap does to both workers in that week — hours, holiday allowance and pay — and to the store's labour cost. On approval the schedule and both weekly summaries are updated. At month end the system prepares a payroll draft and flags minimum-wage problems before payday.

2.4 System environment

This section follows the three parts of the system environment in Chapter 8: the system configuration, the hardware and network configuration and the software configuration.

2.4.1 System configuration

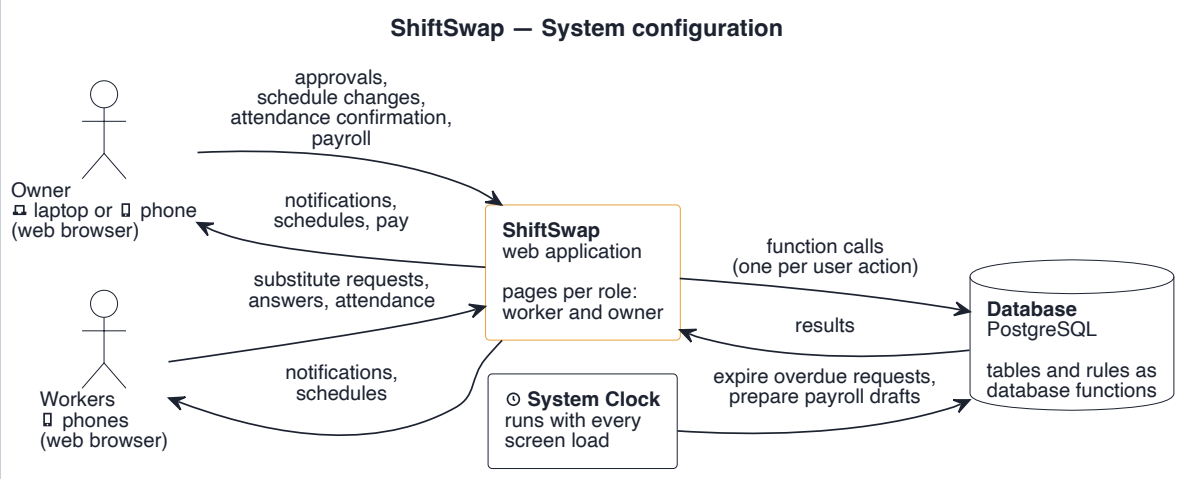


Figure 1. System configuration of ShiftSwap

Figure 1 shows the context of the system. The owner uses ShiftSwap on a laptop or a phone and every worker on their own phone, all in a web browser. The ShiftSwap web application shows the pages of each role and sends every action to the database as one function call. The database (PostgreSQL) stores the eleven tables and carries the business rules as database functions, so that every rule is checked in one place and in one transaction. The System Clock is drawn as an actor because it starts use cases, but it is not a separate process: its duties — ending overdue requests and preparing past payroll drafts — are performed by the database function `run_system_clock()`, which every screen calls first and which calls `expire_overdue_requests()` and the preparation of the drafts. The system uses no external service; notifications stay inside the application (risk R7, Section 10).

2.4.2 Hardware and network configuration

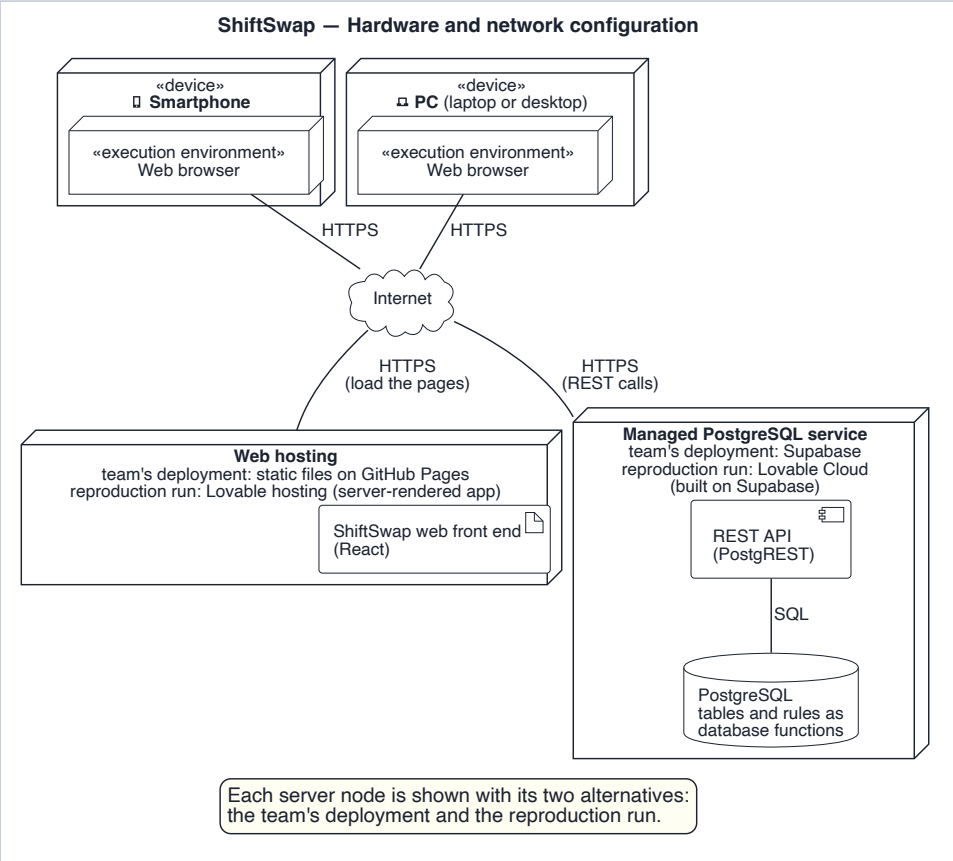


Figure 2. Hardware and network configuration

Table 4. Hardware and network configuration

Node	Team's deployment	Reproduction run of the final prompt	Constraint
Client	Smartphone or PC with a current Chrome, Edge, Safari or Firefox; screens 360 px wide or more		No installation (NFR-06)
Network	Internet, HTTPS only; the public (anon) key of the database API is part of the web page by design		Identity is not verified (NFR-14)

Node	Team's deployment	Reproduction run of the final prompt	Constraint
Web front end and hosting	Static build (Vite) of the React front end on GitHub Pages	Lovable's default React application (TanStack Start, server-rendered) on `*.lovable.app`	No application server of the team's own for business logic
Database	Supabase, managed PostgreSQL (Seoul region), REST API (PostgREST)	Lovable Cloud, built on Supabase's open-source foundation	Free plans pause inactive databases (NFR-16, risk R6)

Both runs use the same database script and functions. The script is expected to run unchanged on Lovable Cloud, which is built on Supabase's open-source foundation [24]; the first trial in Week 9 confirms this. The front ends are generated separately from the same prompt — by Claude Code for the team and by Lovable in a reproduction run — and may differ in layout, so the acceptance checklist of the scenarios S1–S11 is run on both. The team keeps its own deployment on GitHub Pages and on the Supabase project that it already uses for its documents, because developing with Claude Code does not consume Lovable credits, of which the free plan grants five a day and at most 30 a month [24].

2.4.3 Software configuration

Table 5. Software configuration

Part	Software	Use
Client	React with TypeScript, Tailwind CSS, supabase-js (team: built with Vite; reproduction run: Lovable's TanStack Start application)	Pages per role; one database function per action; no business rule in the client
Server	PostgreSQL 15 or later; PL/pgSQL functions (`SECURITY DEFINER`); row-level security; PostgREST	Eleven tables; every rule and state change in a function; tables read-only for the API roles
Vibe-coding tools	Claude Code (Anthropic); Lovable	Claude Code builds and verifies the system from the final prompt; Lovable is the target of the final prompt, with which the instructor re-runs it
Models and tests	PlantUML; Node.js test runner with PGlite (PostgreSQL embedded in the test process); headless Chrome	UML 2.5 diagrams; tests of the database functions (Appendix A); screen checks at 360 and 1280 px

Three decisions shape this configuration. First, every user action is one database function call, so that each action is one transaction; the state of a substitute request changes only by a conditional `UPDATE`, which under PostgreSQL's default isolation makes a concurrent second change find the row already changed [9] (NFR-04). Second, the API roles may only read the tables, and every change passes through a function that checks its input and the state transi-

tion [23] (NFR-14). Third, sign-in is a choice of demo account without a password, because password authentication is outside the scope of the project, as the proposal states; the database guarantees the integrity of the data, not the identity of a user, so all stored data are fictional demo data (NFR-11).

2.5 Demo store

The demo data describe Dalbit Café (달빛카페), a fictional store with four regular employees and the default substitution-attendance policy EXCUSED. They are used in all scenarios, screens and tests of this report.

Table 6. Demo store: owner and workers

ID	Name	Wage (KRW/h)	Contract	Fixed shifts (contract hours per week)
1	Park Jiyoung (owner)	–	–	–
2	Lee Seoyeon	10,500	2026-03-02 to 2027-02-28	Mon, Wed 18:00–23:00 (10)
3	Choi Minho	10,320	2026-06-01 to 2026-12-31	Tue, Thu 18:00–23:00; Sat 12:00–18:00 (16)
4	Jung Hana	10,320	2026-09-01, open-ended; probation to 2026-11-30; not simple labour	Fri 17:00–23:00; Sun 10:00–16:00 (12)
5	Kang Doyun	10,000	2026-08-15 to 2026-11-15; simple labour	Sat 10:00–16:00; Sun 16:00–22:00 (12)

The minimum wage table holds 2025: KRW 10,030 and 2026: KRW 10,320. The work history runs from each contract start to 2026-09-20 with confirmed attendance, and the payroll of March to August 2026 is confirmed. The weeks starting 2026-09-21 and 2026-09-28 are generated, and all shifts of the week of 2026-09-21 have confirmed attendance. One open request exists: Lee Seoyeon's request for Wednesday 2026-09-30 18:00–23:00 (S1). The demo clock stands at Monday 2026-09-28 09:00; it is kept in the database, so the demo data behave the same whenever the system is opened, and the owner's Demo tools move it forward or switch to the real time.

2.6 Actors

The system has three actors. **A1 Worker** (primary, human) is a part-time employee who views their own schedule, requests substitutes, accepts or answers “can't” to requests from co-workers and records attendance. **A2 Owner** (primary, human) is the store owner, who also acts as manager: the owner registers workers and fixed schedules, edits shifts and marks no-shows absent, approves or rejects substitutions, confirms attendance, reviews weekly summaries, confirms monthly payroll and edits settings. **A3 System Clock** (secondary, system) is

a time-triggered actor. It ends open requests at their deadline (UC-11) and prepares the drafts of past months that have no payroll yet (UC-14). It has no process of its own: the database function `run_system_clock()` performs its duties whenever a screen is loaded, and the functions that answer, decide or cancel a request check the deadline themselves, so that no late action can succeed even when no screen was loaded. Its time is the demo clock while it is set, otherwise the database server time in Korea Standard Time.

3. Requirements elicitation

Chapter 8 [7] describes requirements elicitation as work with the stakeholders to find out about the application domain, the services the system should provide and its constraints. Our stakeholders are the store owner, the part-time workers and, indirectly, the labour law that fixes the pay rules. We use four sources.

1. **Domain experience.** Hyewon Kim works part-time in the reference store. Her experience gave the problem statement and the figure of typically 3–5 contacts per substitute request.
2. **Interviews.** Semi-structured interviews with the owner of the reference store and two of Hyewon's co-workers, planned for Week 8. Hyewon Kim will lead each interview and a second member will take notes. Each interview is planned for about 30 minutes.
3. **Document analysis.** The owner's schedule spreadsheet (structure only; no personal data is copied) and the legal sources on minimum wage, weekly holiday, probation and the work of part-time workers beyond their contractual hours [1–5, 20].
4. **Prototype sessions.** The owner and the co-workers will be shown the system of Section 8 and asked to perform one task each (Section 9.2; planned for Week 10).

Table 7. Interview plan

Owner of the reference store — current process, decisions the owner makes, pay calculation
1. How do you learn that a worker cannot come? How far in advance? 2. Whom do you contact first, and how do you decide? How many contacts does one request take? 3. What do you check before you agree to a swap? 4. Where do you record the change? Who else needs to know? 5. How do you decide at month end whether weekly holiday allowance is owed? 6. If a worker hands over a shift by agreement, do you count that week as full attendance? 7. How many regular employees does the store have? Do you pay night or overtime premiums? 8. Do any workers have a probation period or a reduced wage? 9. Which step of the current process costs you the most time? 10. What should the system never do without your approval?
Two co-workers — the worker's view of requesting and taking substitute shifts
1. How often do you need a substitute? For what reasons?

2. How do you find out that a colleague needs a substitute?
3. What makes you accept or decline a request?
4. Has it happened that two people agreed to take the same shift?
5. When would you like the request to stop being open (deadline)?
6. How soon do you usually see and answer a message from a colleague? Would you open an app to check for requests?
7. Which information about you should colleagues not see?
8. How do you check your own hours and pay today?

The questions were derived from P1–P3 and from the open points in the business rules. Question 6 to the owner addresses the only rule where the law leaves room for interpretation (BR-06): its answer decides the default policy for the reference store, and the system keeps both options as a setting in any case. Question 4 to the workers addresses the concurrency requirement (BR-02, NFR-04), question 6 the answer delay behind risk R7, and question 7 privacy (NFR-11). The interviews have not taken place yet; they are planned for Week 8, and their answers will be recorded together with every requirement they change. Until then EXCUSED is a design default that the owner has not yet confirmed.

4. Functional requirements specification

4.1 Functional requirements

Table 8 lists the 24 functional requirements in the format of Chapter 8: requirement number, name, category and description. The name of a requirement is the name of its use case and the category is the package of the use case diagram (Figure 3), so that functional requirements, use cases and scenarios share one vocabulary. The priority is that of the use case except for FR-24, the part-time overtime warning, which has *Should* priority within the *Must* use case UC-06, as the proposal specifies for function F6. FR-21, the notification inbox, serves five use cases and is therefore named by its function. FR-03 belonged to the removed use case UC-02 (Section 1.2); its number is not reused.

Table 8. Functional requirements

Req. number	Req. name	Category	Description	Priority
FR–01	Register worker and fixed schedule	Schedule	The owner shall register a worker with name, phone, hourly wage (KRW), contract start date, contract end date (optional), probation end date (optional), simple-labour flag and active flag.	Must

Req. number	Req. name	Category	Description	Priority
FR–02	Register worker and fixed schedule	Schedule	The owner shall register one or more fixed weekly shifts per worker (weekday, start time, end time); an end time not later than the start time means that the shift ends the next day (BR-05). Fixed shifts of the same worker must not overlap on the weekly time line, including across midnight.	Must
FR–04	Manage weekly schedule	Schedule	The system shall generate the shifts of a week (Monday to Sunday) from the fixed schedules of active workers, only on dates within each worker's contract period; generating the same week again creates no duplicate.	Must
FR–05	Manage weekly schedule	Schedule	The owner shall add, move (date, times, worker) or delete a single shift. A shift with a substitute request or an attendance record cannot be deleted, a shift with an open request cannot be changed, and no change may make two shifts of one worker overlap.	Must
FR–25	Manage weekly schedule	Schedule	The owner shall mark a shift that has started and is still SCHEDULED as ABSENT, whether or not an attendance record exists (a no-show has none).	Must
FR–06	Request substitute	Substitution	A worker shall create a substitute request for one of their own future SCHEDULED shifts, with an optional reason and a reply deadline.	Must
FR–07	Request substitute	Substitution	On creation the system shall compute the eligible candidates (BR-01), create one target per candidate and put a notification in each candidate's inbox, all in one transaction.	Must

Req. number	Req. name	Category	Description	Priority
FR–08	Request substitute	Substitution	If no worker is eligible, the request ends at once as FAILED (BR-13): the requester is told that no co-worker can take the shift and to contact the owner, and the owner receives a notification.	Must
FR–09	Respond to substitute request	Substitution	A candidate shall accept a request or answer “can't”. The first acceptance makes that candidate the acceptor; it is decided by one database function in one statement (BR-02), so a later acceptance is refused with a message that names the first acceptor.	Must
FR–10	Respond to substitute request	Substitution	When a request is accepted, the system shall notify the requester and the owner and close all other pending targets.	Must
FR–22	Respond to substitute request	Substitution	When every target of an open request has answered “can't”, the request ends as FAILED (BR-13); the requester is told to contact the owner and the owner receives a notification that can only be acknowledged.	Must
FR–11	Approve or reject substitution	Substitution	Before the decision the approval screen shall show, for the requester and the acceptor, the weekly contractual hours, the scheduled hours of that week before and after the swap, holiday-allowance eligibility and amount before and after (any change highlighted), and the estimated pay change of the week (base pay, and premium pay in a workplace with five or more regular employees), together with the change in the store's labour cost for that week (BR-16).	Must

Req. number	Req. name	Category	Description	Priority
FR–24	Approve or reject substitution	Substitution	In a workplace with five or more regular employees the approval screen shall warn when the swap would leave a part-time acceptor with more than 12 hours beyond the weekly contractual hours in that week (BR-15); the warning does not block the decision.	Should
FR–12	Approve or reject substitution	Substitution	On approval the system shall reassign the shift to the acceptor (keeping the original worker), recompute both workers' weekly summaries and notify both, in one database function (BR-04). On rejection the shift stays unchanged and both are notified.	Must
FR–13	Record and confirm attendance	Attendance and pay	After a shift has ended, its worker shall record the actual clock-in and clock-out times, which must be plausible (BR-17); the owner shall confirm the record, which makes the shift WORKED, or mark the shift ABSENT. A WORKED or ABSENT shift cannot change status again.	Should
FR–14	Compute weekly hours and holiday allowance	Attendance and pay	The system shall compute, per worker and ISO week, contractual, scheduled and actual hours, perfect attendance, holiday-allowance eligibility and holiday hours (BR-05 to BR-07). A week is not recomputed when the month of its Sunday has a confirmed payroll.	Must
FR–15	Generate and confirm monthly payroll	Attendance and pay	The owner shall generate a monthly payroll draft per worker (base pay, holiday pay, premium pay, total; BR-08 to BR-11) and confirm it; a confirmed payroll is read-only.	Should

Req. number	Req. name	Category	Description	Priority
FR–16	Generate and confirm monthly payroll	Attendance and pay	The payroll shall warn when a worker's hourly wage is below the applicable minimum (BR-09), apply the probation minimum only when BR-10 holds, and block confirmation until every warning of the month is acknowledged.	Should
FR–23	Prepare past payroll drafts automatically	Attendance and pay	When the System Clock runs, every past month without a payroll row gets a DRAFT computed as in FR-15, and the owner receives one notification per month; existing rows are never overwritten and nothing is confirmed automatically (BR-14).	Should
FR–17	Manage workplace settings and minimum wage table	Administration	The owner shall edit the store name, the number of regular employees, the substitution-attendance policy (BR-06) and the yearly minimum wage rows.	Should
FR–18	Expire overdue substitute request	Substitution	A request still open when its deadline or its shift start has passed shall become EXPIRED, its pending targets closed and the requester and the owner notified; this is evaluated whenever a screen is loaded and inside the functions that answer, decide or cancel a request (BR-12).	Must
FR–19	Cancel own substitute request	Substitution	A requester shall cancel their own open request; pending targets are closed and every target that was pending or accepted is notified.	Should

Req. number	Req. name	Category	Description	Priority
FR–20	Sign in by role	Administration	Users shall sign in by choosing a demo account without a password (password authentication is out of scope); the screen states that all data are fictional demo data shared by everyone who opens the application, and the interface shows only the menus of the chosen role. The owner's Demo tools set the demo clock and restore the demo data.	Must
FR–21	Notification inbox (UC-04 to UC-06, UC-11, UC-12)	Substitution	Every user shall have an in-app notification inbox, newest first, with the unread count in the header.	Must

4.2 Dependencies of increment 1

Increment 1 must be a usable system on its own, so no *Must* use case may need a *Should* use case. Table 9 lists what each *Must* use case needs and how increment 1 provides it.

Table 9. What the *Must* use cases need and how increment 1 provides it

Must **use case**	**Needs**	**Provided in increment 1 by**
UC-08 Compute weekly hours and holiday allowance	Whether each past shift was worked (BR-06 b)	BR-06: a past shift that the owner has not marked ABSENT counts as worked. Marking a no-show absent is FR-25 of UC-03 (*Must*); recording clock times (UC-07) is not needed.
UC-08	The substitution-attendance policy	Seeded default EXCUSED; editing it is UC-10 (*Should*).
UC-06 Approve or reject substitution	Hourly wages, number of regular employees, minimum wages	Wages from UC-01 (*Must*); 4 regular employees and the minimum wages of 2025 and 2026 are seeded defaults.
UC-11 Expire overdue substitute request	The current time	The database function `app_now()`.
UC-04, UC-05	Notifications	FR-21 (*Must*).

UC-14, the automatic payroll draft (new since the proposal, Table 3), is *Should* because it reuses the payroll computation of UC-09, which is *Should*.

4.3 Business rules

The business rules are the domain requirements (Chapter 8) that several functional requirements share. They are stated once here and referenced by number; each one is implemented by the database functions and tested by the test group of Appendix A.

Table 10. Business rules

ID	Rule
BR-01	**Eligible candidates.** A worker W is eligible for a request on shift S if and only if W is an active worker, W is not the requester, the date of S lies within W's contract period, and no shift of W overlaps S on the time line of BR-05 (so a shift that ends after midnight on the previous day counts). Every eligible co-worker is asked.
BR-02	**First acceptance wins.** The accept function changes the request with one statement, `UPDATE … SET status = 'ACCEPTED', acceptor = W WHERE id = R AND status = 'REQUESTED' AND deadline > now AND shift start > now`. If it changes no row, the acceptance is refused (“already taken” when another candidate is the acceptor, otherwise “request closed”). The candidate must hold a pending target and must not have an overlapping shift; the overlap is checked again at approval. On success all other pending targets are closed.
BR-03	**Request validity.** Only the shift's current worker can request; the shift must be SCHEDULED and start in the future; at most one open (REQUESTED or ACCEPTED) request per shift; the deadline must be after now and not later than the shift start.
BR-04	**Approval effect.** Approve: the shift's worker becomes the acceptor, the shift keeps its first worker as original worker, and the request becomes APPROVED. Reject: the request becomes REJECTED and the shift is unchanged. Only an ACCEPTED request can be decided, and only before its deadline and shift start.
BR-05	**Time line and week.** A shift occupies the interval from its date and start time to its end time on the same date, or on the next date when the end time is not later than the start time. A shift lasts between 30 minutes and 16 hours. It belongs to the date and the ISO week (Monday 00:00 to Sunday 24:00) on which it starts, even when it ends after midnight. Two shifts overlap if and only if their intervals overlap; fixed shifts are compared on the weekly time line in the same way (Monday 22:00–02:00 overlaps Tuesday 01:00–05:00).

ID	Rule
BR-06	**Holiday-allowance eligibility (주휴).** W is eligible for an ISO week if and only if (a) W's weekly contractual hours C, the sum of W's fixed-schedule hours, are at least 15, and (b) perfect attendance: no shift of the week that W originally held and still holds is ABSENT. A shift that has not been marked ABSENT counts as worked, so the owner only has to mark no-shows; the result for the current week is a projection. A shift that W gave away by an approved substitution counts by the workplace policy: EXCUSED (default; the working day was changed by agreement, so attendance is not broken) or ABSENT (attendance is broken). Shifts W took as a substitute do not change C. (Labor Standards Act Art. 55 ①, Art. 18 ③, Enforcement Decree Art. 30 ① [3, 4]; the law averages 15 hours over four weeks, and the fixed schedule is constant, so the average equals C.)
BR-07	**Holiday-allowance hours.** Hours = min(C, 40) / 40 × 8, rounded to two decimals; weekly holiday pay = hours × hourly wage, rounded down to the won.
BR-08	**Premium pay.** Applies only if the workplace has five or more regular employees (Labor Standards Act Art. 11, 56 [3]; Fixed-term and Part-time Employees Act Art. 3 ②, Art. 6, Enforcement Decree Art. 2 and Annex 1 [20]). Then +50 % of the wage for (i) hours of a work date beyond 8, (ii) hours of an ISO week beyond 40 not counted in (i), (iii) night hours, that is the part of a shift's interval between 22:00 and 06:00 of the next day on the time line (a shift 22:00–02:00 has 4 night hours), and (iv) for a part-time worker, hours beyond the weekly contractual hours not counted in (i) or (ii). The night premium is added to the others. System rules for (iv), design decisions where the law does not fix the method: (a) a worker with 0 < C < 40 is part-time; (b) the test is weekly against C; (c) the hours of a week are taken in time order, and every hour after the first C hours is beyond contract; (d) an hour that meets more than one of (i), (ii) and (iv) counts once; (e) the night premium is separate. In a monthly payroll the days of a week outside the month only decide where the first C (and the first 40) hours end. The worker's consent is outside the system; the 12-hour limit is checked before approval (BR-15).
BR-09	**Minimum-wage check.** Warn if the hourly wage is below the minimum of the year, multiplied by 0.9 if BR-10 applies. The payroll still uses the worker's wage; the warning blocks confirmation of the month until the owner acknowledges it. Re-generating a draft requires the acknowledgement again.
BR-10	**Probation reduction.** The minimum may be reduced to 90 % only if all hold: a contract of at least one year (2026-09-01 to 2027-08-31 counts as one year) or an open-ended contract, a date within three months of the contract start and not after the probation end, and work that is not simple labour (Minimum Wage Act Art. 5 ②, Enforcement Decree Art. 3 [5]). In a monthly payroll the conditions are evaluated on the first day of the month, or on the contract start if later.

ID	Rule
BR–11	**Monthly payroll.** A worker gets a row for the month if they have a paid shift or holiday pay in it, or if they are active and their contract overlaps the month. Hours are summed in minutes and converted once. Base pay = minutes of the month's shifts × wage / 60, rounded down: WORKED shifts with their recorded times, SCHEDULED shifts with their scheduled times (the row is then flagged estimated), ABSENT shifts with zero. Holiday pay = sum of the weekly holiday pay of the weeks whose Sunday falls in the month. Premium pay = premium minutes × wage × 0.5 / 60, rounded down. Total = base + holiday + premium.
BR–12	**Expiry.** An open request whose deadline or shift start has passed becomes EXPIRED; pending targets are closed and the requester and the owner are notified. This is evaluated whenever a screen is loaded, and the functions that answer, decide or cancel a request check the deadline themselves, so a late action is refused even if no screen was loaded; the expiry is then stored by the next screen load.
BR–13	**No taker.** A request ends as FAILED when it has no eligible candidate at creation, or when every target has answered “can't”. The requester and the owner are notified; the owner's only action is to acknowledge. A FAILED request cannot be accepted.
BR–14	**Automatic payroll drafts.** When the System Clock runs, every month from the first contract month to the month before now that has no payroll row gets a DRAFT computed as by FR-15, and the owner is notified once per month. No row is overwritten and nothing is confirmed.
BR–15	**Part-time weekly limit warning.** With five or more regular employees and a part-time acceptor ($0 < C < 40$), let S be the acceptor's scheduled hours in the week of the shift after the swap. If $S - C > 12$, the approval screen shows a warning with $S - C$ (“13 h beyond contract, limit 12 h”); exactly 12 hours gives no warning. The warning does not block the approval (Fixed-term and Part-time Employees Act Art. 6 [20]).
BR–16	**Swap cost estimate.** For an accepted request on shift S in week W, the approval preview computes, for the requester and the acceptor, the week's pay before and after S moves from the requester to the acceptor: base pay (scheduled minutes of the shifts not marked ABSENT × wage / 60), premium pay (BR-08 applied to the same shifts of W, zero below five regular employees) and holiday pay (BR-06 and BR-07 under the workplace policy). The change of each worker is “after – before” per component, and the change in the store's labour cost is the sum of both workers' changes.
BR–17	**Attendance plausibility.** Attendance is recorded only after the shift's scheduled end. Clock-in is placed on the day on which it falls within two hours before the scheduled start and four hours after the scheduled end, and clock-out after it (on the next day when it is not later than the clock-in); the recorded interval must lie within that range and last between 30 minutes and 16 hours. A 21:30 clock-in on a 22:00–02:00 shift is accepted; 23:00 to 18:00 on an 18:00–23:00 shift is not.

4.4 Use-case descriptions

Each use case is described in the same structure: actors, preconditions, main flow, alternative and exception flows, postconditions, business rules and related functional requirements. Every use case except UC-11, UC-13 and UC-14 requires a signed-in user (UC-13); this precondition is not repeated. Each step that changes data is one call of the database function named in brackets.

Table 11. UC-01 Register worker and fixed schedule

Actors	Owner
Preconditions	A worker is hired, or a worker's contract or fixed schedule changes.
Main flow	1. The owner opens *Staff* and chooses to add a worker. 2. The owner enters name, phone, hourly wage, contract start and end, optional probation end, the simple-labour flag and the "currently employed" (active) flag. 3. The system validates the input and stores the worker (`save_worker`). 4. Under "Fixed shifts" the owner picks weekdays and sets start and end times; an end time not later than the start time means that the shift ends the next day. 5. The system checks that no two fixed shifts of the worker overlap on the weekly time line and stores them (`save_fixed_schedule`). 6. The system shows the fixed schedule and the weekly contractual hours ("Saved contract: 16 h a week" for Choi Minho).
Alternative / exception flows	1a. The owner edits an existing worker or deactivates them; an inactive worker is no longer included in generated weeks or candidate lists. 3a. A required field is missing or the wage is not a positive integer: the system names the field and the fix; nothing is stored. 5a. Two fixed shifts overlap, also across midnight (Monday 22:00–02:00 and Tuesday 01:00–05:00): the system refuses and names the conflicting shift.
Postconditions	The worker and the fixed shifts are stored; the worker is included in the next generated week within the contract period.
Business rules / FR	BR-05 (contractual hours feed BR-06) | FR-01, FR-02

Table 12. UC-03 Manage weekly schedule

Actors	Owner (the worker views the week read-only)
Preconditions	At least one active worker with a fixed schedule exists.
Main flow	1. The owner opens *Schedule* and navigates to a week (Monday to Sunday). 2. If the week has no shifts yet, the system generates it (`generate_week`): one SCHEDULED shift per fixed shift of every active worker, on dates within the worker's contract period, skipping slots that already produced a shift. 3. The system shows the week with seven day columns, shifts ordered by start time; a shift that ends after midnight is shown on the day it starts.
Alternative / exception flows	1a. A worker opens *Schedule*: the week is read-only, the worker's own shifts are highlighted, and *Find a substitute* is offered on an own future shift. 2a. The week is already generated: nothing is created twice. 3a. The owner adds, moves or deletes a single shift (`add_shift`, `update_shift`, `delete_shift`): the system refuses a change that makes two shifts of one worker overlap, a change of a shift with an open request, and the deletion of a shift with a request or attendance history. 3b. A shift has started and the worker did not come: the owner opens the shift and chooses *Mark absent* (`mark_absent`); the shift becomes ABSENT, with or without an attendance record, and the worker's week is recomputed (BR-06).

Postconditions	The shifts of the week exist exactly once per fixed shift; edited and absent shifts are stored.
Business rules / FR	BR-05, BR-06 | FR-04, FR-05, FR-25

Table 13. UC-04 Request substitute

Actors	Worker (requester); System (candidate selection)
Preconditions	The worker is the current worker of a SCHEDULED shift that starts in the future and has no open request.
Main flow	1. The worker chooses *Find a substitute* (on *Schedule*, on *Swaps*, or on the next-shift card of *Home*). 2. The system leads the worker through three steps: the shift, an optional reason, and the reply deadline, pre-filled with the earlier of now + 24 h and the shift start. 3. The system finds the eligible candidates «include» *Find eligible candidates* (BR-01) and shows them on the last step (“Check the deadline and send”). 4. The worker chooses *Send swap request*. 5. In one function call (`create_sub_request`) the system checks BR-03 and stores the request as REQUESTED, one PENDING target per candidate and one notification per candidate. 6. The request appears on *Swaps* under “Your requests” at the step “Requested”; on *Schedule* the shift is marked “Looking for cover”.
Alternative / exception flows	2a. A shift that already has an open request, or has started, is not offered (BR-03). 5a. BR-03 fails at sending time (for example the deadline is no longer valid): the system names the cause and the fix; nothing is stored. 5b. No co-worker is eligible: the request ends at once as FAILED; the requester is told to contact the owner, and the owner receives a notification to acknowledge (BR-13). Extension points: deadline reached (UC-11); requester withdraws (UC-12).
Postconditions	The request is REQUESTED with one PENDING target and one unread notification per candidate, or FAILED.
Business rules / FR	BR-01, BR-03, BR-13 | FR-06, FR-07, FR-08, FR-21

Table 14. UC-05 Respond to substitute request

Actors	Worker (candidate)
Preconditions	The candidate has a PENDING target on a REQUESTED request.
Main flow	1. The candidate opens *Home*; the card under “Swap requests for you” shows the requester, the shift, the reason, the reply deadline and the effect on the candidate's own week, with *Accept* and *Can't*. 2. The candidate chooses *Accept*. 3. One function call (`accept_sub_request`) first checks that the deadline and the shift start have not passed (BR-12), then changes the request with one conditional statement from REQUESTED to ACCEPTED with this candidate as acceptor, sets the candidate's target to ACCEPTED and closes all other pending targets (BR-02). 4. The system notifies the requester and the owner, and the closed candidates. 5. On the requester's *Swaps* screen the request moves to the step “Accepted”.

Alternative / exception flows	2a. The candidate chooses *Can't* (`decline_sub_request`): the target becomes DECLINED and nobody is notified; the request stays open for the others. 2b. This was the last pending target and every target answered "can't": the request ends as FAILED (BR-13). 3a. Another candidate accepted first: the conditional statement changes no row and the acceptance is refused with a message that names the first acceptor (FR-09). 3b. The request has expired, failed or been cancelled: the acceptance is refused with that reason. 3c. The candidate meanwhile has an overlapping shift: the acceptance is refused.
Postconditions	At most one acceptor per request; no PENDING target remains once the request is ACCEPTED.
Business rules / FR	BR-02, BR-12, BR-13 | FR-09, FR-10, FR-22, FR-21

Table 15. UC-06 Approve or reject substitution

Actors	Owner
Preconditions	A request is ACCEPTED.
Main flow	1. The owner opens *Home*; the section "Swaps waiting for approval" lists every ACCEPTED request as a card. 2. For the requester and the acceptor the system computes the preview (`approval_preview`): contractual hours, scheduled hours of the week before and after, holiday-allowance eligibility and amount before and after, and the pay change of the week; and the change in the store's labour cost (FR-11, BR-16). 3. The card shows one line per worker with these changes and highlights any change of eligibility. In a workplace with five or more regular employees it also shows the premium change and, for a part-time acceptor beyond 12 hours over contract, the warning of BR-15 (FR-24). 4. The owner chooses *Approve*. 5. One function call (`decide_sub_request`) first checks that the deadline and the shift start have not passed, then sets the shift's worker to the acceptor, keeps the original worker, sets the request to APPROVED, recomputes both workers' weeks «include» UC-08, and notifies both workers.
Alternative / exception flows	3a. The warning of BR-15 is shown: it does not block the decision, because the worker's consent and the owner's decision are outside the system. 4a. The owner chooses *Reject*: the request becomes REJECTED, the shift is unchanged and both workers are notified. 5a. The request expired before the decision (BR-12): the decision is refused and the list is refreshed.
Postconditions	Approved: the shift belongs to the acceptor and both weekly summaries are current. Rejected: no schedule change.
Business rules / FR	BR-04, BR-06, BR-07, BR-08, BR-15, BR-16 | FR-11, FR-12, FR-24, FR-21

Table 16. UC-07 Record and confirm attendance

Actors	Worker, Owner
Preconditions	The shift has ended (worker) or started (owner marking absent).

Main flow	1. The worker opens *Me*; the "Attendance" section lists the worker's shifts with their state. 2. For a shift that has ended, the worker enters the actual clock-in and clock-out times (`record_attendance`). 3. The system checks plausibility (BR-17) and stores the record as unconfirmed. 4. The owner reviews the unconfirmed records on *Home* and confirms one (`confirm_attendance`); the shift becomes WORKED. 5. The system recomputes the worker's week (UC-08).
Alternative / exception flows	2a. The shift has not ended yet: recording is refused. 3a. The times are implausible (for example 23:00 to 18:00 on a 18:00–23:00 shift, or a clock-in long before the shift): the system refuses and explains the allowed range; a clock-out after midnight on a shift that ends after midnight is accepted. 4a. The worker did not work: the owner marks the shift ABSENT (as in UC-03, 3b).
Postconditions	The shift is WORKED or ABSENT; the weekly summary reflects it.
Business rules / FR	BR-06 (input), BR-17 | FR-13

Table 17. UC-08 Compute weekly hours and holiday allowance

Actors	Owner; System (when included by UC-06 or UC-09)
Preconditions	The shifts of the week are generated.
Main flow	1. The owner opens *Pay*, tab *Weekly*, and picks a week. 2. For each worker with a shift that week the system computes contractual hours (fixed schedule), scheduled hours (shifts held) and actual hours (confirmed attendance) (`week_summary`). 3. The system determines perfect attendance: a shift marked ABSENT breaks it; an unmarked past shift counts as worked; a shift given away by approved substitution counts by the policy (EXCUSED or ABSENT). 4. The system determines eligibility (BR-06) and holiday hours and pay (BR-07) and stores one summary row per worker and week. 5. The system shows one card per worker with the eligibility, the holiday pay if eligible, and a short reason if not (for example "Contract 10 h a week · under 15 h" or "absent on 9/29").
Alternative / exception flows	2a. The month of the week's Sunday has a confirmed payroll: the stored rows are shown and not recomputed. 2b. The week is not generated: the system says so; the owner generates it in UC-03 first.
Postconditions	The weekly summaries of the week are current (one per worker and week).
Business rules / FR	BR-05, BR-06, BR-07 | FR-14

Table 18. UC-09 Generate and confirm monthly payroll

Actors	Owner
Preconditions	The minimum wage of the month's year exists.

Main flow	1. The owner opens *Pay*, tab *Monthly*, and picks a month. 2. The system builds the draft (`generate_payroll`) after recomputing the weeks whose Sunday falls in the month «include» UC-08. 3. For each worker the system computes hours and base pay, holiday pay, premium pay and total (BR-08, BR-11) and checks the minimum wage (BR-09, BR-10). 4. The system shows the rows as DRAFT with status chips (Draft, Estimated, Probation minimum 90 %, Below minimum wage). 5. The owner acknowledges each minimum-wage warning (`acknowledge_min_wage`) and confirms the month (`confirm_payroll`); the rows become CONFIRMED and read-only.
Alternative / exception flows	2a. The month already has DRAFT rows: they are replaced; a confirmed month is not changed. 3a. Attendance is missing for some shifts: their scheduled time is used and the row is flagged “Estimated”. 5a. A warning is not acknowledged: confirmation is refused and the screen states how many warnings remain.
Postconditions	One payroll row per worker and month, DRAFT or CONFIRMED.
Business rules / FR	BR-08 to BR-11 | FR-15, FR-16

Table 19. UC-10 Manage workplace settings and minimum wage table

Actors	Owner
Preconditions	The store's data, its number of regular employees, its policy or the legal minimum wage changed.
Main flow	1. The owner opens *Settings*. 2. The system shows the store name, the number of regular employees, the policy for handed-over shifts (“Do not count as absence” = EXCUSED, the default, or “Count as absence” = ABSENT) and the minimum wage by year. 3. The owner edits values, adds or removes minimum-wage rows and saves (`update_settings`).
Alternative / exception flows	3a. The number of regular employees is not a non-negative integer, or a year appears twice: the system refuses and explains.
Postconditions	Later weekly summaries use the new policy; later payroll drafts and previews use the new employee count and minimum wages.
Business rules / FR	BR-06 (policy), BR-08 (threshold), BR-09 (minimum wage) | FR-17

Table 20. UC-11 Expire overdue substitute request

Actors	System Clock
Preconditions	A request is REQUESTED or ACCEPTED.
Main flow	1. A screen is loaded; before it reads its data the application calls `run_system_clock()`. 2. The function reads the current time (`app_now()`: the demo clock while it is set, otherwise the server time in Korea). 3. It finds the open requests whose deadline or shift start has passed. 4. For each it sets the request to EXPIRED and closes the pending targets. 5. It notifies the requester and the owner.

Alternative / exception flows	1a. No screen is loaded before somebody answers, decides or cancels: the function that does it checks the deadline itself and refuses the late action (“request closed”); the expiry is stored by the next screen load. 3a. No request is overdue: nothing changes.
Postconditions	No open request has a past deadline when a screen shows it; the shift stays with the requester.
Business rules / FR	BR-12 | FR-18, FR-21

Table 21. UC-12 Cancel own substitute request

Actors	Worker (requester)
Preconditions	The worker's request is REQUESTED or ACCEPTED.
Main flow	1. The worker opens *Swaps*; under “Your requests” each request shows its progress (Requested → Accepted → Approved). 2. The worker chooses *Cancel request* on an open request (`cancel_sub_request`). 3. The system sets the request to CANCELLED and closes all pending targets. 4. The system notifies every candidate that was pending and the acceptor, if any.
Alternative / exception flows	2a. The request is already APPROVED, REJECTED, FAILED, EXPIRED or CANCELLED: no *Cancel request* button is shown.
Postconditions	The request is CANCELLED; the shift stays with the requester.
Business rules / FR	– | FR-19, FR-21

Table 22. UC-13 Sign in by role (demo account selection, no password)

Actors	Worker, Owner
Preconditions	The application is open in a browser.
Main flow	1. The system shows the demo page: a notice that all data are fictional demo data shared by everyone who opens the application; a three-step tour (see Lee Seoyeon's request, accept as Choi Minho, approve as owner Park Jiyoung); and the list of all accounts (owner and four workers) with their roles. There are no passwords, because password authentication is outside the scope of the project. 2. The user starts the tour or chooses an account. 3. The system keeps the choice for the browser session and shows only the menus of that role: *Home, Schedule, Swaps, Me* for a worker; *Home, Schedule, Staff, Pay, Settings* for the owner.
Alternative / exception flows	3a. The user signs out: the choice is cleared and the demo page is shown. 3b. The owner opens *Demo tools*: moves the demo clock (+1 hour, +1 day, a chosen date) or switches to the real time (`set_demo_clock`), or restores the demo data and the demo clock (`reset_demo_data`) after a confirmation.
Postconditions	A signed-in user with role OWNER or WORKER.
Business rules / FR	– | FR-20 (NFR-11, NFR-14)

Table 23. UC-14 Prepare past payroll drafts automatically

Actors	System Clock (the owner receives the notification)
Preconditions	The current month is later than the first contract month.

Main flow	1. A screen is loaded and `run_system_clock()` runs (as in UC-11). 2. For every month from the first contract month to the month before now that has no payroll row, the function generates the DRAFT «include» UC-09. 3. It notifies the owner once per drafted month (“September payroll draft is ready”).
Alternative / exception flows	2a. Every past month already has rows: nothing changes; existing rows are never overwritten and nothing is confirmed.
Postconditions	Every past month has payroll rows; the owner reviews, acknowledges and confirms them in UC-09.
Business rules / FR	BR-14 | FR-23

5. Non-functional requirements specification

Non-functional requirements are classified as in Chapter 8 into *product* requirements (how the delivered system must behave), *organisational* requirements (which follow from the team's and the course's policies) and *external* requirements (which come from outside the system and the project, here mainly law). Table 24 uses the format of Chapter 8: requirement number, type, name and specification, with the verification method added. The numbers NFR-01 to NFR-16 are kept from the proposal so that its references remain valid.

Table 24. Non-functional requirements

Req. number	Req. type	Req. name	Specification	Verification
Product requirements				
NFR–01	Usability	Quick answer	A candidate can accept a request in at most 2 clicks from *Home* or the inbox.	UI walkthrough
NFR–02	Usability	Phone and desktop	All screens are usable at 360 px (phone) and 1280 px (desktop) width without horizontal scrolling.	Browser check at both widths
NFR–03	Performance	Response time	With 20 workers and 12 weeks of shifts, every screen shows its data within 1 second after the database answers; a screen needs at most three database calls.	Timed load with seeded data

Req. number	Req. type	Req. name	Specification	Verification
NFR-04	Reliability	First acceptance wins	Acceptance, decision, cancellation and expiry each run as one database function in one transaction; the state change is one conditional `UPDATE … WHERE status = …`, so that of two concurrent acceptances exactly one succeeds.	Function tests TC-05x; test with two connections
NFR-05	Reliability	Persistence and reset	Data persist in the database; the owner can restore the demo data, which also resets the demo clock.	Function test TC-19x; manual test
NFR-06	Portability	Browsers	Runs in current Chrome, Edge, Safari and Firefox without installation.	Manual test
NFR-13	Usability	Task-first screens	Each role starts on a *Home* screen that lists the items waiting for that person with the action on each item; each screen has one primary action; an action that changes hours or pay shows its effect before the user commits; the interface is in Korean with an English alternative.	Walkthrough; headless check
NFR-14	Security	Writes only through functions	Row-level security is enabled on every table with read-only policies; the API roles have no right to insert, update or delete; every change goes through a database function that runs with the rights of the database role that owns it (`SECURITY DEFINER`), with an empty search path, and validates its input and the state transition. Identity is not verified (demo accounts, FR-20).	Function tests TC-16x; REST test

Req. number	Req. type	Req. name	Specification	Verification
NFR–15	Efficiency	Fresh screens	A change made by one user appears on the open screens of the others within 10 seconds (refresh on an interval and when the window regains focus).	Two-browser check
NFR–16	Dependability	Availability	Free plans pause inactive databases. The team's database is kept active by a scheduled read every six hours (GitHub Actions); because GitHub disables the schedules of a public repository after 60 days without activity [22], the deployed URL is also checked weekly until grading ends and before every submission (risk R6) [21].	Workflow log; check log
Organisational requirements				
NFR–07	Delivery	Deadline	Working system deployed by Week 14 (2026-12-04).	Deployed URL
NFR–08	Implementation	Platform and method	Web front end in React with TypeScript; PostgreSQL with all business rules as database functions; built by vibe coding with Claude Code from the final prompt; the final prompt also builds the system in Lovable, whose Lovable Cloud provides the same PostgreSQL.	Repository; reproduction run
NFR–09	Standards	Models and names	Models follow UML 2.5 notation [8]; code identifiers are in English.	Review
External requirements				

Req. number	Req. type	Req. name	Specification	Verification
NFR–10	Legislative	Labour law	Pay rules follow the Minimum Wage Act (2026 minimum hourly wage KRW 10,320), the Labor Standards Act (Art. 18 ③, Art. 55, Enforcement Decree Art. 30; Art. 11 and 56 for the premium threshold) and the Fixed-term and Part-time Employees Act (Art. 6); where the law leaves the method open, the system's rule is stated as a design decision (BR-06, BR-08, BR-15, BR-16).	Function tests TC-08x, TC-09x, TC-15x, TC-17x
NFR–11	Privacy	Minimum disclosure on screen	The screens show a worker's phone number and wage only to the owner and to that worker (principle of minimum disclosure of the Personal Information Protection Act). Because sign-in is a choice of demo account and the database can be read with the public key, this protection is screen-level only; therefore all data are fictional demo data, which the sign-in screen states.	Role test; design review
NFR–12	Ethical	No location data	The system collects no location data; attendance is self-reported and owner-confirmed.	Design review

Two of these requirements shape the design directly. NFR-04 means that a candidate's acceptance cannot be handled by reading the request and then writing it in a second step: the status check and the change are one statement in one database function. NFR-14 means that, without an application server of the team's own, the database protects the data itself: the API may only read, and every change passes through a function that checks the rules. Because sign-in remains a choice of demo account (FR-20), NFR-14 guarantees integrity but not identity, and NFR-11 is therefore limited to the screens and to fictional data.

6. Scenarios

Chapter 8 defines a scenario as a real-life example of how the system is used. Each scenario below is given in the format of Chapter 8 — title, actors, initial condition, final condition, exceptional case and the process step by step — with the use cases and rules it exercises. Each alternative sequence is a separate scenario, as Chapter 8 recommends. All scenarios use the demo store of Section 2.5; unless stated otherwise, the demo clock stands at Monday 2026-09-28 09:00 and the policy is EXCUSED. The numbers are those that the database functions compute; the same numbers are expected values of the test cases (Appendix A).

Table 25. Scenario S1 — Lee Seoyeon requests a substitute for her midterm exam

Actors	Lee Seoyeon (worker, requester)
Initial condition	Seoyeon works Monday and Wednesday 18:00–23:00. She has a midterm exam on Wednesday 2026-09-30 and cannot work that evening.
Final condition	One open request with three candidates, each with an unread notification. This is the state of the demo data.
Exceptional case	A deadline after the shift start (2026-09-30 18:00) is refused with the explanation that the deadline must not be later than the shift start.
Process	1. Seoyeon signs in and opens *Schedule* for the week of 2026-09-28; her own shifts are highlighted. 2. She chooses *Find a substitute* and picks her Wednesday 30 September 18:00–23:00 shift. 3. She enters the reason “Midterm exam” and the deadline 2026-09-29 21:00. 4. The last step shows the candidates by BR-01: Choi Minho, Jung Hana and Kang Doyun, none of whom has a shift overlapping Wednesday 18:00–23:00. 5. She sends the request; the system checks BR-03 and stores the request as REQUESTED with three pending targets and three notifications, in one function call. 6. On *Schedule* her Wednesday shift is marked “Looking for cover”.
Related	UC-04 | FR-06, FR-07, FR-21 | BR-01, BR-03

Table 26. Scenario S2 — Choi Minho accepts first; Kang Doyun is refused

Actors	Choi Minho, Kang Doyun (workers, candidates)
Initial condition	The final condition of S1.
Final condition	The request is ACCEPTED with Minho as acceptor; the other targets are closed; the shift still belongs to Seoyeon until the owner decides.
Exceptional case	Doyun presses *Accept* at almost the same moment as Minho. Both function calls run; the second waits for the first to finish and then finds the status no longer REQUESTED, so its conditional statement changes no row and Doyun sees “Choi Minho accepted first” (BR-02, NFR-04, Figure 11).
Process	1. Minho signs in; the bell shows one unread notification and *Home* lists the request under “Swap requests for you”. 2. The card shows Seoyeon's Wednesday shift, the reason, the reply deadline and the effect on his own week: 16 h → 21 h, holiday allowance unchanged (KRW 33,024). 3. He chooses *Accept*, one tap; the request becomes ACCEPTED with Minho as acceptor, and the targets of Hana and Doyun are closed. 4. Seoyeon and the owner are notified that Minho accepted; Hana and Doyun are notified that the request is closed. 5. On Seoyeon's *Swaps* screen the request moves to the step “Accepted”.
Related	UC-05 | FR-09, FR-10 | BR-02

Table 27. Scenario S3 — The owner sees the cost of the swap and approves it

Actors	Park Jiyoung (owner)
Initial condition	The final condition of S2. The store has four regular employees, so no premium applies.
Final condition	Minho holds the Wednesday shift; Seoyeon is recorded as its original worker; both weekly summaries are current; the request is APPROVED.
Exceptional case	The owner chooses *Reject*: the request becomes REJECTED, the shift stays with Seoyeon and both are notified.
Process	1. The owner signs in; *Home* reads "1 thing to handle today" and lists the swap under "Swaps waiting for approval". 2. The card shows the preview of Table 28: Seoyeon's week 10 h → 5 h, pay −52,500; Minho's week 16 h → 21 h, pay +51,600, holiday allowance unchanged at 33,024; the store's labour cost for the week −900. 3. She chooses *Approve*; in one function call the shift's worker becomes Minho, Seoyeon is kept as original worker, the request becomes APPROVED, both weeks are recomputed and both workers are notified. 4. On *Schedule* the Wednesday shift shows Minho, with Seoyeon struck through beneath.
Related	UC-06, UC-08 | FR-11, FR-12 | BR-04, BR-06, BR-07, BR-16

Table 28. Approval preview of S3 (week of 2026-09-28, policy EXCUSED, four regular employees)

Worker	Contract h	Scheduled h	Holiday allowance	Base pay change (KRW)	Premium change	Total change
Lee Seoyeon (requester)	10	10 → 5	not eligible → not eligible (contract under 15 h)	−52,500	0	−52,500
Choi Minho (acceptor)	16	16 → 21	eligible → eligible, 33,024 → 33,024	+51,600	0	+51,600
Store labour cost for the week						**−900**

Minho's holiday allowance stays 16 / 40 × 8 = 3.2 hours, that is 3.2 × 10,320 = KRW 33,024, because a substitute shift does not change contractual hours. Seoyeon's 5 hours cost 5 × 10,500 = 52,500 and Minho's 5 × 10,320 = 51,600, so the swap saves the store KRW 900 in this week. Under the policy EXCUSED no eligibility changes; S7 shows the same preview under ABSENT, and S8 with five regular employees.

Table 29. Scenario S4 — A request expires at its deadline

Actors	System Clock; Lee Seoyeon and the owner (notified)
Initial condition	The final condition of S1: the request is REQUESTED with deadline 2026-09-29 21:00. In this variant no candidate answers.
Final condition	The request is EXPIRED, its targets are closed, and the Wednesday shift stays with Seoyeon. She may create a new request with a deadline before 2026-09-30 18:00.

Exceptional case	Minho presses *Accept* on a screen that was opened before the deadline: the accept function checks the deadline itself and refuses the acceptance (“request closed”). The same holds for an accepted request that the owner has not decided before the deadline.
Process	1. The owner moves the demo clock in *Demo tools* from Monday 09:00 to Tuesday 2026-09-29 21:00. 2. The next screen load runs the System Clock function: the deadline has passed, so the request becomes EXPIRED and the pending targets are closed. 3. Seoyeon and the owner are notified that the request expired; Minho, Hana and Doyun no longer see it.
Related	UC-11 | FR-18 | BR-12

Table 30. Scenario S5 — Jung Hana asks for a Sunday substitute and nobody can take it

Actors	Jung Hana (requester); Lee Seoyeon, Choi Minho, Kang Doyun (candidates); the owner
Initial condition	Hana cannot work Sunday 2026-10-04 10:00–16:00.
Final condition	The request is FAILED; Hana knows to contact the owner, and the owner has acknowledged the case.
Exceptional case	One candidate accepts before the last “can't”: the request becomes ACCEPTED and continues as in S2 and S3.
Process	1. Hana requests a substitute for the Sunday shift; the candidates are Seoyeon, Minho and Doyun (Doyun's Sunday 16:00–22:00 shift does not overlap 10:00–16:00). 2. Seoyeon, Minho and Doyun each answer *Can't*; the first two answers notify nobody. 3. With the third answer every target is DECLINED, so the request ends as FAILED (BR-13). 4. Hana sees “Every co-worker said they can't cover this shift. Please contact the owner.” The owner's *Home* shows a card whose only action is *Got it*.
Related	UC-04, UC-05 | FR-08, FR-22 | BR-13

Table 31. Scenario S6 — September payroll is reviewed and confirmed

Actors	Park Jiyoung (owner)
Initial condition	The demo clock stands at 2026-10-01 09:00 and the September draft has been prepared automatically (S10). The store has four regular employees.
Final condition	The September payroll is CONFIRMED and read-only.
Exceptional case	The owner tries to confirm before acknowledging the warning: confirmation is refused with “Acknowledge every minimum-wage warning (1) before confirming.” Correcting a wage requires changing it in UC-01 and generating the draft again before confirmation.

Process	1. The owner opens *Pay*, tab *Monthly*, September 2026 (Table 32). 2. Premium pay is 0 for every worker, because the store has fewer than five regular employees (BR-08). 3. Holiday pay counts the four weeks whose Sunday falls in September (Sundays 6, 13, 20 and 27 September). Minho is eligible in each (16 h contract): 4 × 33,024 = 132,096. Seoyeon (10 h), Hana (12 h) and Doyun (12 h) are below 15 hours. 4. Doyun's wage of KRW 10,000 is below the 2026 minimum of KRW 10,320. BR-10 does not apply to him, because his contract (2026-08-15 to 2026-11-15) is shorter than one year and his work is simple labour, so his row shows a minimum-wage warning (BR-09). 5. For Hana the probation minimum applies (open-ended contract, September within three months of 2026-09-01 and before the probation end 2026-11-30, not simple labour): 10,320 × 0.9 = 9,288; her wage is above it, so there is no warning. 6. The shifts of 28 to 30 September have no recorded attendance yet, so Seoyeon's and Minho's rows are flagged “Estimated” (BR-11). 7. The owner acknowledges Doyun's warning and confirms; all rows become CONFIRMED.
Related	UC-09 | FR-15, FR-16 | BR-07, BR-09, BR-10, BR-11

Table 32. September 2026 payroll draft of S6 (KRW)

Worker	Hours	Base pay	Holiday pay	Premium	Total	Flags
Lee Seoyeon	45	472,500	0	0	472,500	Estimated
Choi Minho	69	712,080	132,096	0	844,176	Estimated
Jung Hana	48	495,360	0	0	495,360	Probation minimum 9,288
Kang Doyun	48	480,000	0	0	480,000	Below minimum wage
All workers					**2,292,036**	

Table 33. Scenario S7 — Policy ABSENT: a handed-over shift costs the holiday allowance

Actors	Park Jiyoung (owner); Choi Minho (requester); Jung Hana (acceptor)
Initial condition	The owner changes the policy for handed-over shifts in *Settings* from “Do not count as absence” (EXCUSED) to “Count as absence” (ABSENT). Minho cannot work Saturday 2026-10-03 12:00–18:00.
Final condition	Depends on the owner's decision. If approved, Hana holds the Saturday shift and Minho loses the holiday allowance of the week of 2026-09-28.
Exceptional case	Under the default policy EXCUSED the same swap leaves Minho eligible, with 3.2 holiday hours (KRW 33,024).

Process	1. Minho requests a substitute; BR-01 selects Lee Seoyeon and Jung Hana; Kang Doyun is not eligible because his Saturday 10:00–16:00 shift overlaps. 2. Hana accepts. 3. The owner's approval card shows the preview of Table 34: under ABSENT the given-away shift breaks Minho's perfect attendance, so his eligibility changes from eligible to not eligible and is highlighted. 4. The owner decides with this information.
Related	UC-10, UC-04, UC-05, UC-06 | FR-11, FR-17 | BR-06, BR-16

Table 34. Approval preview of S7 (week of 2026-09-28, policy ABSENT, four regular employees)

Worker	Contract h	Scheduled h	Holiday allowance	Base pay change (KRW)	Holiday pay change	Total change
Choi Minho (requester)	16	16 → 10	**eligible → not eligible**	−61,920	−33,024	−94,944
Jung Hana (acceptor)	12	12 → 18	not eligible → not eligible	+61,920	0	+61,920
Store labour cost for the week						**−33,024**

Table 35. Scenario S8 — A swap takes a part-time worker beyond the weekly limit

Actors	Park Jiyoung (owner); Choi Minho (acceptor); Lee Seoyeon (requester)
Initial condition	The request of S1 is REQUESTED. The owner sets the number of regular employees to 5 in *Settings* and adds a shift for Minho on Friday 2026-10-02 10:00–18:00. Minho's week then has 24 scheduled hours against 16 contractual hours; he is part-time (0 < 16 < 40).
Final condition	If the owner approves, Minho holds 29 scheduled hours in the week, 13 of them beyond contract; his contractual hours and his holiday allowance (3.2 h) are unchanged. If she rejects, the shift stays with Seoyeon.
Exceptional case	With the Friday shift 10:00–17:00 instead, the preview shows 23 → 28 hours, exactly 12 hours beyond contract, and no warning. With four regular employees there is neither a warning nor a premium.
Process	1. Minho accepts Seoyeon's Wednesday request. 2. The owner's card shows Minho's week going from 24 to 29 hours and the warning “13 h beyond contract, limit 12 h” (29 − 16 = 13 > 12; BR-15), and the premium estimate of Table 36. 3. The warning does not block the decision; whether Minho consents is outside the system. 4. If the 29 hours are worked as scheduled, BR-08 (iv) takes them in time order: Tuesday 5, Wednesday 5 (the swapped shift), Thursday 5, Friday 8 and Saturday 6. The 16 contractual hours are reached one hour into Friday, so 7 hours on Friday and 6 on Saturday, 13 hours in total, are beyond contract, and October's payroll includes 13 × 10,320 × 0.5 = KRW 67,080 for them; the night hours are paid separately.
Related	UC-10, UC-03, UC-05, UC-06 | FR-11, FR-24 | BR-08, BR-15, BR-16

Table 36. Approval preview of S8 (week of 2026-09-28, five regular employees)

Worker	Scheduled h	Premium hours before → after	Base pay change (KRW)	Premium change	Total change
Lee Seoyeon (requester)	10 → 5	night 2 → 1	−52,500	−5,250	−57,750
Choi Minho (acceptor)	24 → 29	beyond contract 8 → 13, night 2 → 3	+51,600	+30,960	+82,560
Store labour cost for the week					**+24,810**

With five regular employees the same swap that saved KRW 900 in S3 costs the store KRW 24,810 more in this week, because Minho's extra hours are beyond his contract ($5 \times 10{,}320 \times 0.5 = 25{,}800$) and one of them is a night hour ($10{,}320 \times 0.5 = 5{,}160$), while Seoyeon loses one night hour ($10{,}500 \times 0.5 = 5{,}250$). This is the information that the default policy alone could not show.

Table 37. Scenario S9 — A shift that ends after midnight

Actors	Park Jiyoung (owner)
Initial condition	The store needs a closing shift on Friday 2026-10-02 from 22:00 to 02:00.
Final condition	Kang Doyun has a shift on Friday 22:00–02:00 of 4 hours, in the week of 2026-09-28.
Exceptional case	The owner then tries to add a shift for Doyun on Saturday 2026-10-03 01:00–05:00: it is refused because it overlaps the Friday shift on the time line (OVERLAP), although the two shifts are on different dates.
Process	1. The owner opens *Schedule*, chooses *Add shift*, picks Friday 2 October, 22:00 to 02:00 and Kang Doyun. 2. Because the end time is earlier than the start time, the shift ends on Saturday at 02:00; it lasts 4 hours and belongs to Friday and to the week of 2026-09-28 (BR-05). 3. The board shows the shift in Friday's column, marked as ending the next day. 4. In a workplace with five regular employees all 4 hours are night hours (BR-08 (iii)); with four there is no premium.
Related	UC-03 | FR-05 | BR-05, BR-08

Table 38. Scenario S10 — The September payroll draft is prepared automatically

Actors	System Clock; Park Jiyoung (owner, notified)
Initial condition	The demo data: payroll is confirmed up to August 2026, and September has no payroll row.
Final condition	September 2026 has a DRAFT row per worker and the owner is notified that the draft is ready.
Exceptional case	The demo clock is moved further, to 2026-12-02: the drafts of September, October and November are all prepared, one notification each; rows that already exist are not changed.

Process	1. The owner moves the demo clock to 2026-10-01 09:00. 2. The next screen load runs the System Clock function; September has no payroll row, so its draft is generated as in UC-09. 3. The owner's *Home* shows "September payroll draft is ready" with *Review payroll*, which opens the draft of S6.
Related	UC-14 | FR-23 | BR-14

Table 39. Scenario S11 — The owner marks a no-show absent

Actors	Park Jiyoung (owner)
Initial condition	The demo clock stands at Tuesday 2026-09-29 23:30. Choi Minho did not come to his Tuesday 18:00–23:00 shift and recorded nothing.
Final condition	The shift is ABSENT; Minho's week of 2026-09-28 has no perfect attendance and no holiday allowance (33,024 → 0).
Exceptional case	The owner opens a shift that has not started yet: *Mark absent* is not offered.
Process	1. The owner opens *Schedule*, week of 2026-09-28, and taps Minho's Tuesday shift. 2. The shift has started and is still SCHEDULED, so the sheet offers *Mark absent*; she chooses it. 3. The shift becomes ABSENT and Minho's week is recomputed: "No holiday allowance this week: absent on 9/29".
Related	UC-03, UC-08 | FR-25, FR-14 | BR-06

The eleven scenarios exercise nine of the thirteen use cases; UC-01, UC-07, UC-12 and UC-13 have no scenario of their own. Table 40 shows where each use case is exercised, in a scenario or in a test case of Appendix A.

Table 40. Use cases, scenarios and test cases

Use case	Scenarios	Test cases (Appendix A)
UC-01 Register worker and fixed schedule	(setting for S8, S9)	TC-013, TC-19C
UC-03 Manage weekly schedule	S8, S9, S11	TC-011, TC-012, TC-018, TC-019, TC-01A, TC-01B, TC-01C
UC-04 Request substitute	S1, S5, S7	TC-014, TC-021–TC-023, TC-132
UC-05 Respond to substitute request	S2, S5	TC-051–TC-055, TC-131
UC-06 Approve or reject substitution	S3, S7, S8	TC-061–TC-063, TC-151–TC-153, TC-171–TC-173
UC-07 Record and confirm attendance	–	TC-181–TC-185 (including a clock-out after midnight)
UC-08 Compute weekly hours and holiday allowance	S3, S7, S11	TC-081–TC-088
UC-09 Generate and confirm monthly payroll	S6	TC-101–TC-103, TC-111–TC-114

Use case	Scenarios	Test cases (Appendix A)
UC-10 Manage workplace settings and minimum wage table	S7, S8	TC-19B, TC-163
UC-11 Expire overdue substitute request	S4	TC-121–TC-124
UC-12 Cancel own substitute request	–	TC-125
UC-13 Sign in by role	(every scenario starts with it)	TC-19A
UC-14 Prepare past payroll drafts automatically	S10	TC-141, TC-142

7. UML diagrams for requirements analysis

This section presents the analysis models, drawn in UML 2.5 notation [8]. Following Chapter 2 and the object-oriented analysis of the course, they show the system from several perspectives: the external perspective (use case diagram), the behavioural perspective (activity, state machine and one analysis sequence diagram) and the structural perspective (analysis class diagram with entity and control classes). The design class diagram and the remaining sequence diagrams belong to the design and are given in the final report.

7.1 Use case diagram

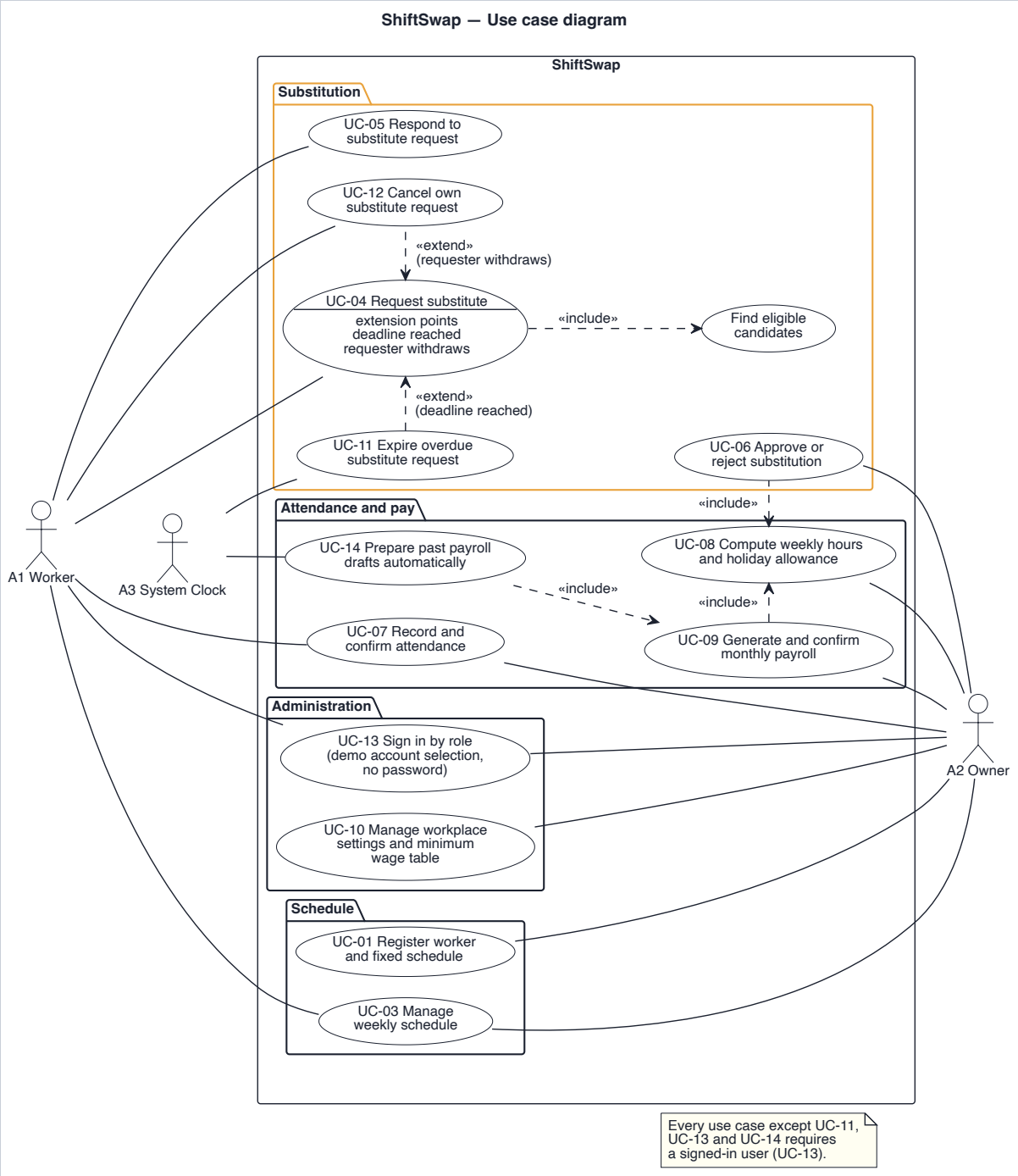


Figure 3. Use case diagram of ShiftSwap

Figure 3 shows the three actors and the thirteen use cases in four packages, which are also the categories of the functional requirements (Table 8). The Worker and the Owner are primary actors; the System Clock is a secondary actor that starts UC-11 and UC-14. UC-04 includes *Find eligible candidates*, the place of rule BR-01. UC-06 and UC-09 include UC-08, because every approval recomputes two workers' weeks and every payroll uses the weekly holiday allowance; UC-14 includes UC-09. UC-11 extends UC-04 at the extension point “deadline reached” and UC-12 at “requester withdraws”. Every use case except UC-11, UC-13 and UC-

14 requires a signed-in user (UC-13), which the diagram states in a note. The nine main functions of the proposal were UC-01 to UC-09; with UC-02 removed (Section 1.2), eight remain. The Worker is associated with UC-03 because workers view the week, read-only.

7.2 Activity diagrams

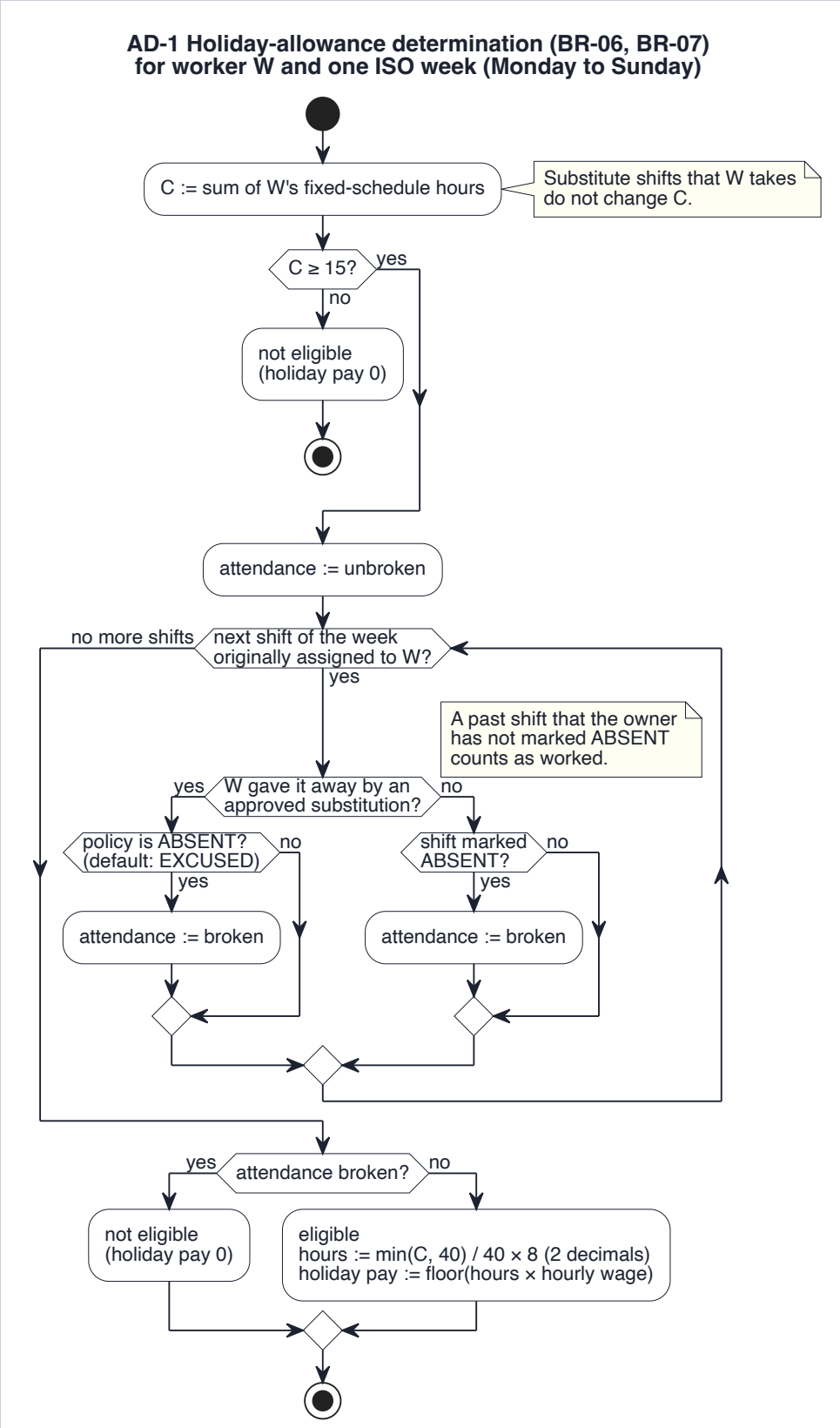


Figure 4. Activity diagram AD-1: holiday-allowance determination (BR-06, BR-07)

Figure 4 shows the weekly holiday-allowance decision for one worker. The first decision is the 15-hour test on contractual hours. The second is perfect attendance: a shift the worker still holds breaks it only if it is marked ABSENT — a past shift without a mark counts as worked, which is what lets increment 1 decide eligibility without attendance records (Table 9). A shift the worker gave away by an approved substitution follows the workplace policy. If the worker is eligible, the holiday hours are min(C, 40) / 40 × 8. The diagram makes the policy variant of S7 and the no-show of S11 visible, and it shows why a substitute shift never enters the 15-hour test.

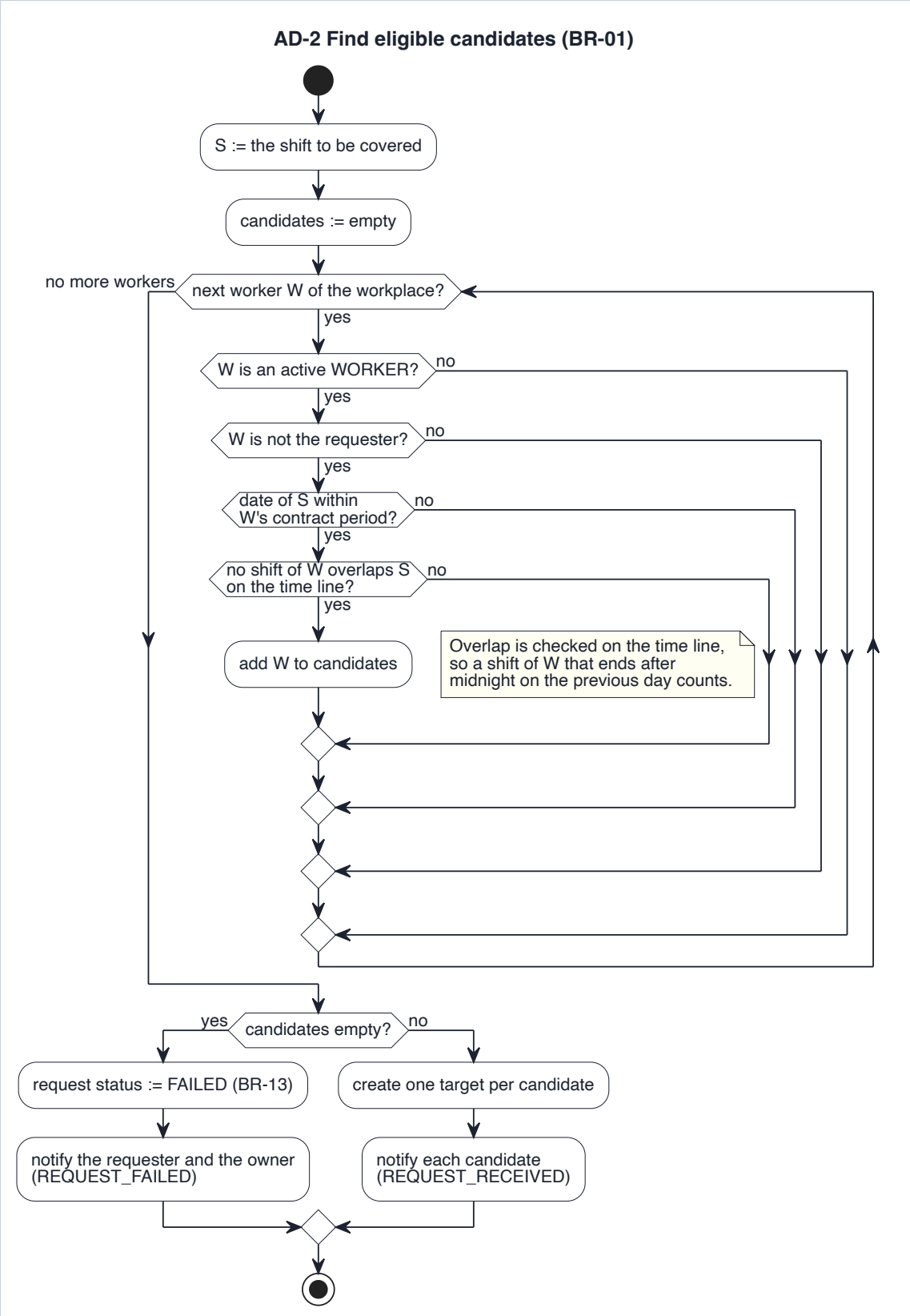


Figure 5. Activity diagram AD-2: candidate selection (BR-01, BR-13)

Figure 5 shows how candidates are selected when a request is created. For each worker the system tests the four conditions of BR-01 in order: active worker, not the requester, the date within the contract period, and no shift overlapping the requested shift on the time line. A

worker who fails a test is skipped. After the loop the diagram branches on the number of candidates: with one or more, one target and one notification per candidate are created; with none, the request ends as FAILED and the requester and the owner are notified. Scenario S1 follows the first branch with three candidates, S7 shows the overlap test rejecting Kang Doyun, and S5 reaches FAILED through the answers instead (BR-13).

7.3 State machine diagrams

Chapter 8 requires that states in a requirements model are domain-significant, that all sequences from the scenarios are allowed and that prohibited sequences are not. The three diagrams below are read against scenarios S1 to S11 with these rules. Each transition is labelled with the database function that makes it; no other path changes the state. Expiry is made by `expire_overdue_requests()`, which the System Clock function `run_system_clock()` calls whenever a screen is loaded.

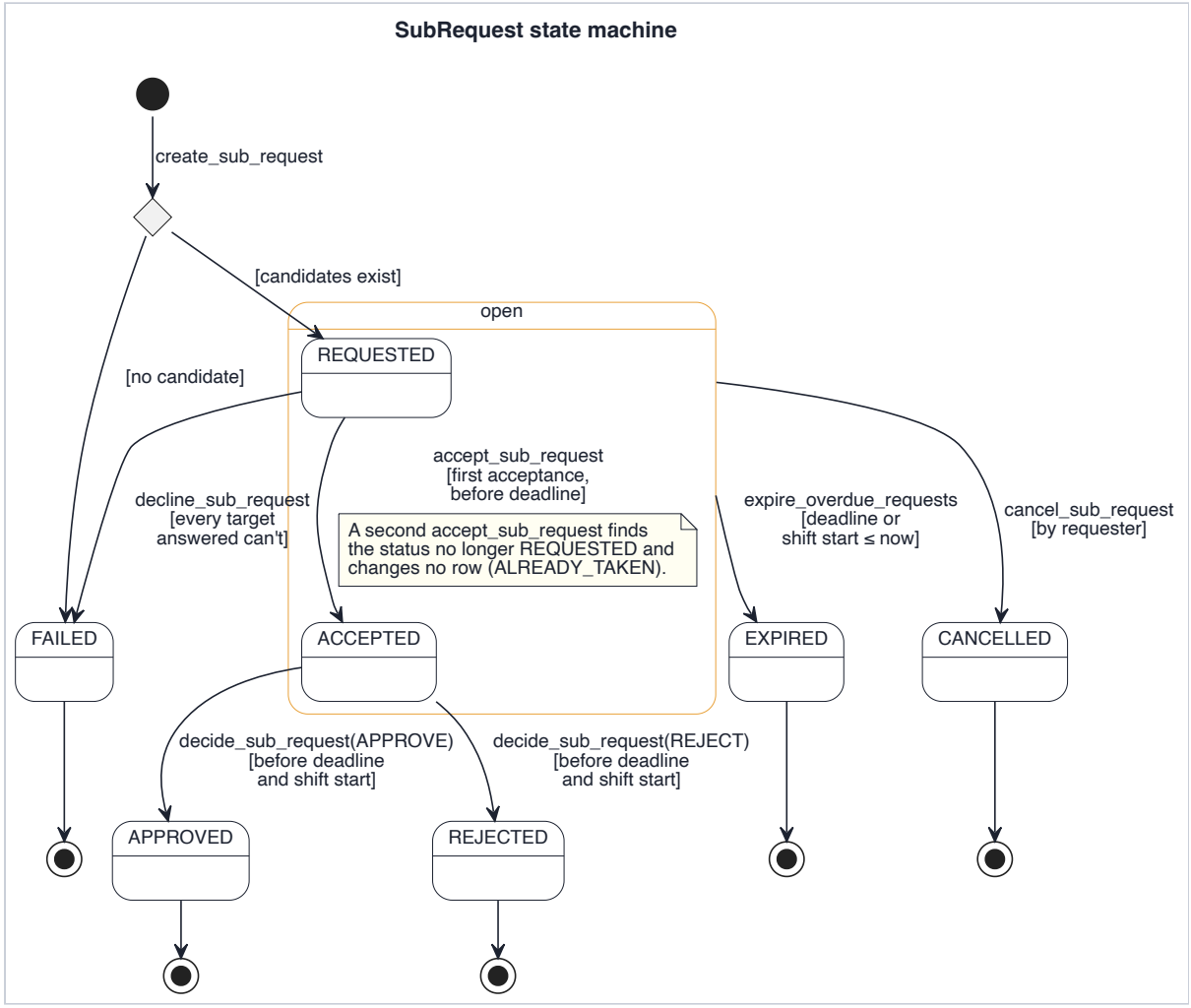


Figure 6. State machine diagram of SubRequest

Figure 6 shows the life of a substitute request. It starts as REQUESTED, or ends at once as FAILED when there is no candidate. The first acceptance moves it to ACCEPTED (UC-05); when every candidate answers “can't” it becomes FAILED. The owner's decision moves an accepted request to APPROVED or REJECTED (UC-06). From REQUESTED or

ACCEPTED it can also become EXPIRED (UC-11) or CANCELLED (UC-12). The seven states include five final states. The diagram also shows what is prohibited: there is no transition from REQUESTED directly to APPROVED, so the owner cannot approve a request nobody accepted (BR-04), and there is no transition out of ACCEPTED on a second acceptance, which is the state-machine form of BR-02. Scenarios S1–S5 each follow one path of this diagram.

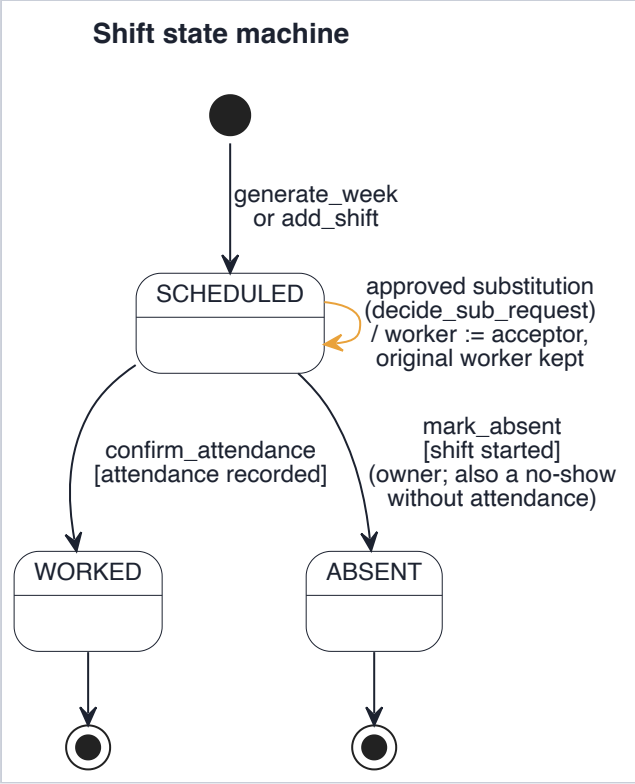


Figure 7. State machine diagram of Shift

Figure 7 shows that a shift is SCHEDULED when generated or added and becomes WORKED when the owner confirms its attendance (UC-07) or ABSENT when the owner marks it so (UC-03 or UC-07) — the latter also without an attendance record, as in S11. Reassignment on approval (BR-04) is a self-transition on SCHEDULED: the worker changes, the state does not. A shift that is WORKED or ABSENT can no longer be requested, because BR-03 requires a SCHEDULED shift in the future.

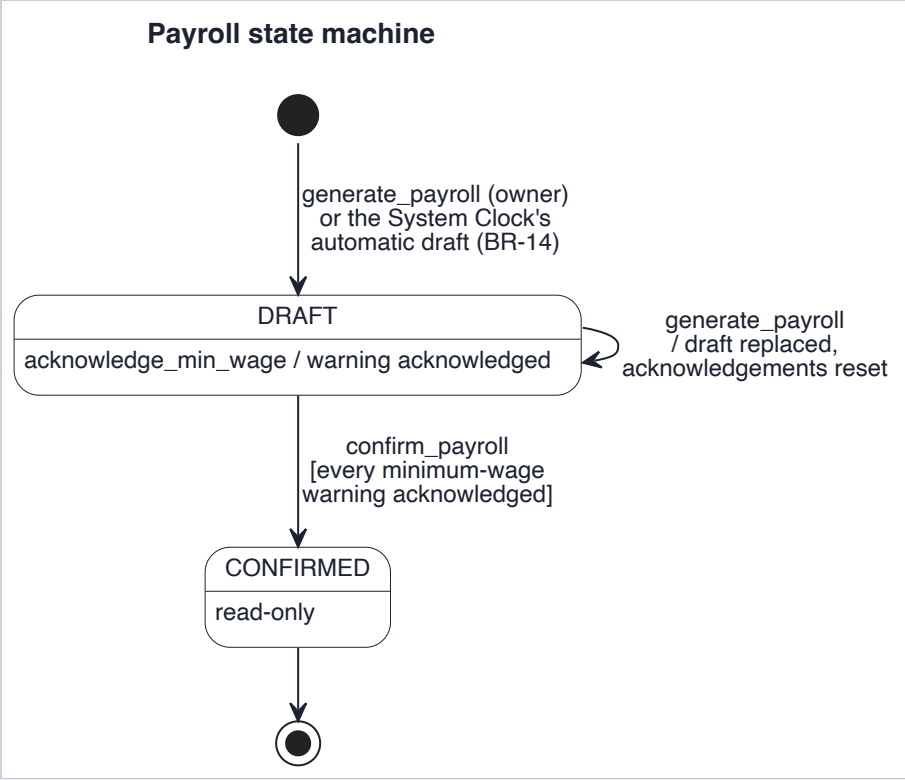


Figure 8. State machine diagram of Payroll

Figure 8 shows the two payroll states. A generated row — by the owner or automatically by the System Clock (UC-14) — is DRAFT. Generating the month again replaces DRAFT rows. Confirmation is guarded: it is possible only when every minimum-wage warning of the month has been acknowledged (BR-09). A CONFIRMED row is final and read-only (FR-15), which S6 ends with.

7.4 Analysis class diagrams

The analysis model uses the two kinds of classes of the course's object-oriented analysis: entity classes for the data of the domain and control classes for the use cases. To keep the diagrams readable, the entity classes and the control classes are drawn in two figures; the boundary classes are the pages of Section 8.

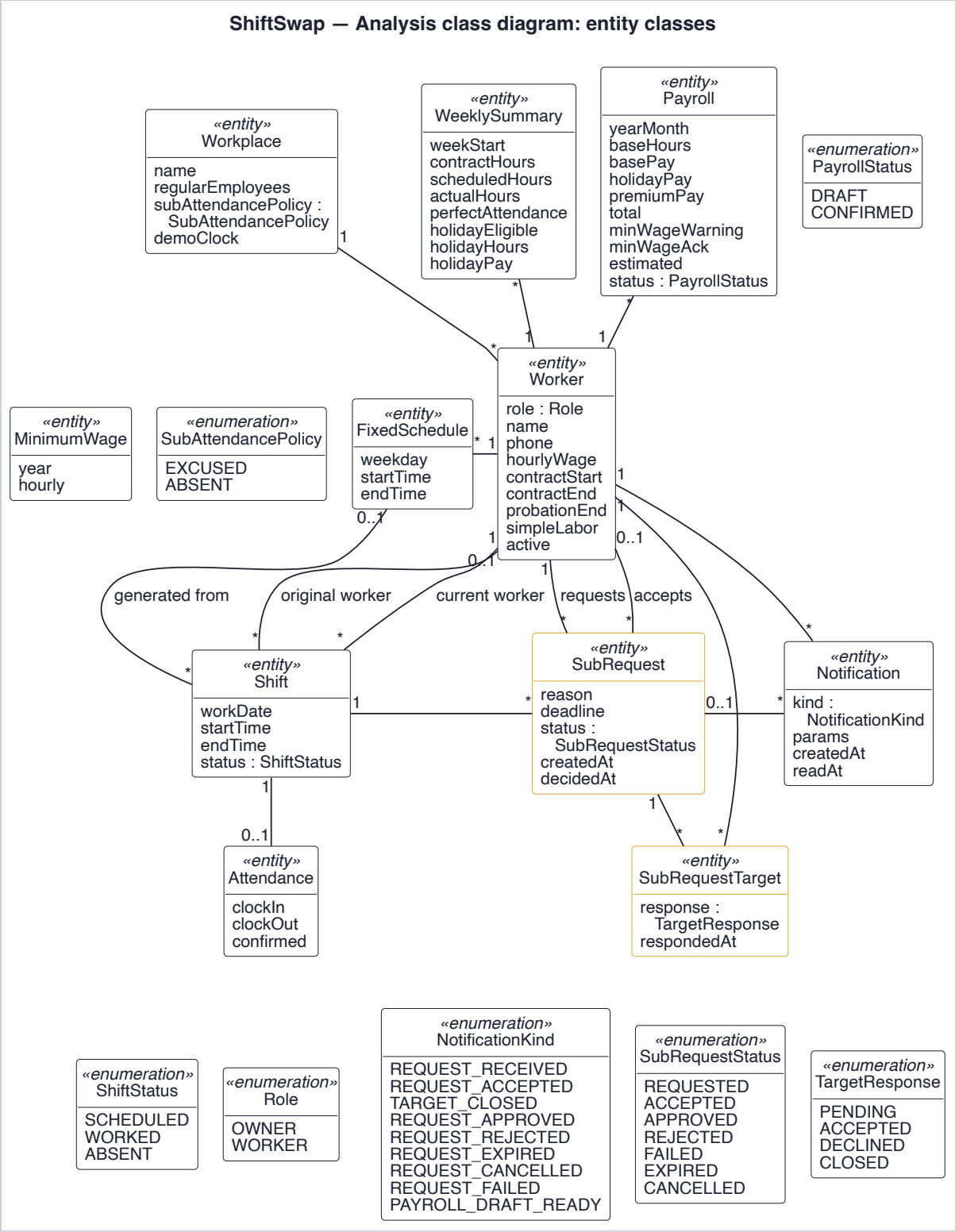


Figure 9. Analysis class diagram: entity classes

Figure 9 shows the eleven entity classes and their associations: a Workplace has Workers; a Worker has FixedSchedule entries, which define the contractual hours; Shifts are generated from FixedSchedule entries and refer to their current worker and, after a swap, to their original worker; a SubRequest belongs to one Shift, has one requester, at most one acceptor and one SubRequestTarget per candidate; Attendance is at most one per Shift; WeeklySummary

and Payroll are derived records per worker and week or month; MinimumWage holds one row per year; a Notification belongs to one recipient and optionally to one request. The enumerations give the states of Section 7.3. Each entity class corresponds to one database table, so the system has eleven tables, above the course minimum of four.

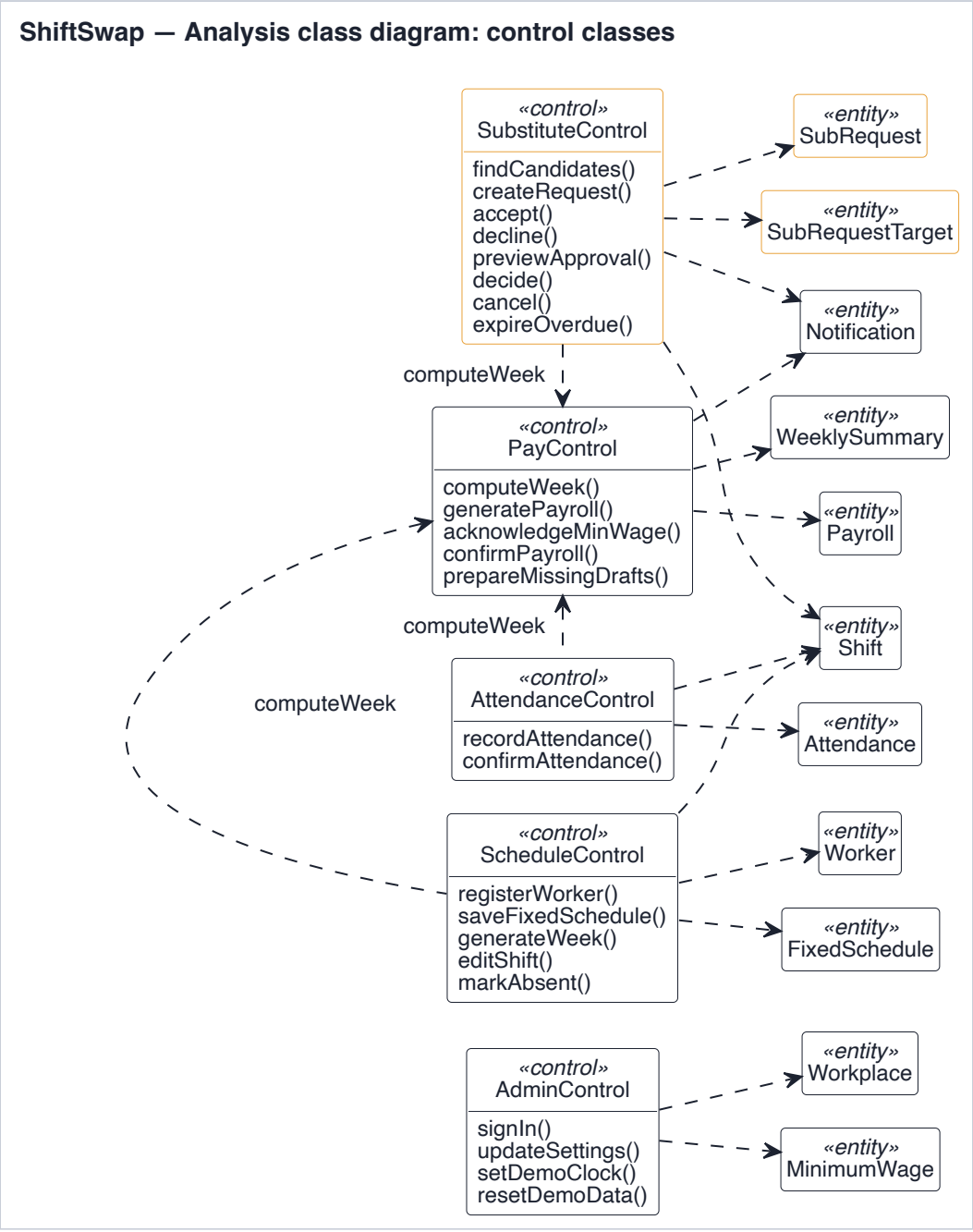


Figure 10. Analysis class diagram: control classes

Figure 10 shows the five control classes, which group the steps of the use cases: ScheduleControl (UC-01, UC-03), SubstituteControl (UC-04 to UC-06, UC-11, UC-12), AttendanceControl (UC-07), PayControl (UC-08, UC-09, UC-14) and AdminControl (UC-10, UC-13). The dependencies show which entities each one changes; ScheduleControl, SubstituteControl and AttendanceControl use PayControl to recompute a week after marking a shift absent, approving a swap or confirming attendance (UC-06 «include» UC-08). In the

design each control operation becomes one database function, so the pages never change an entity directly.

7.5 Analysis sequence diagram

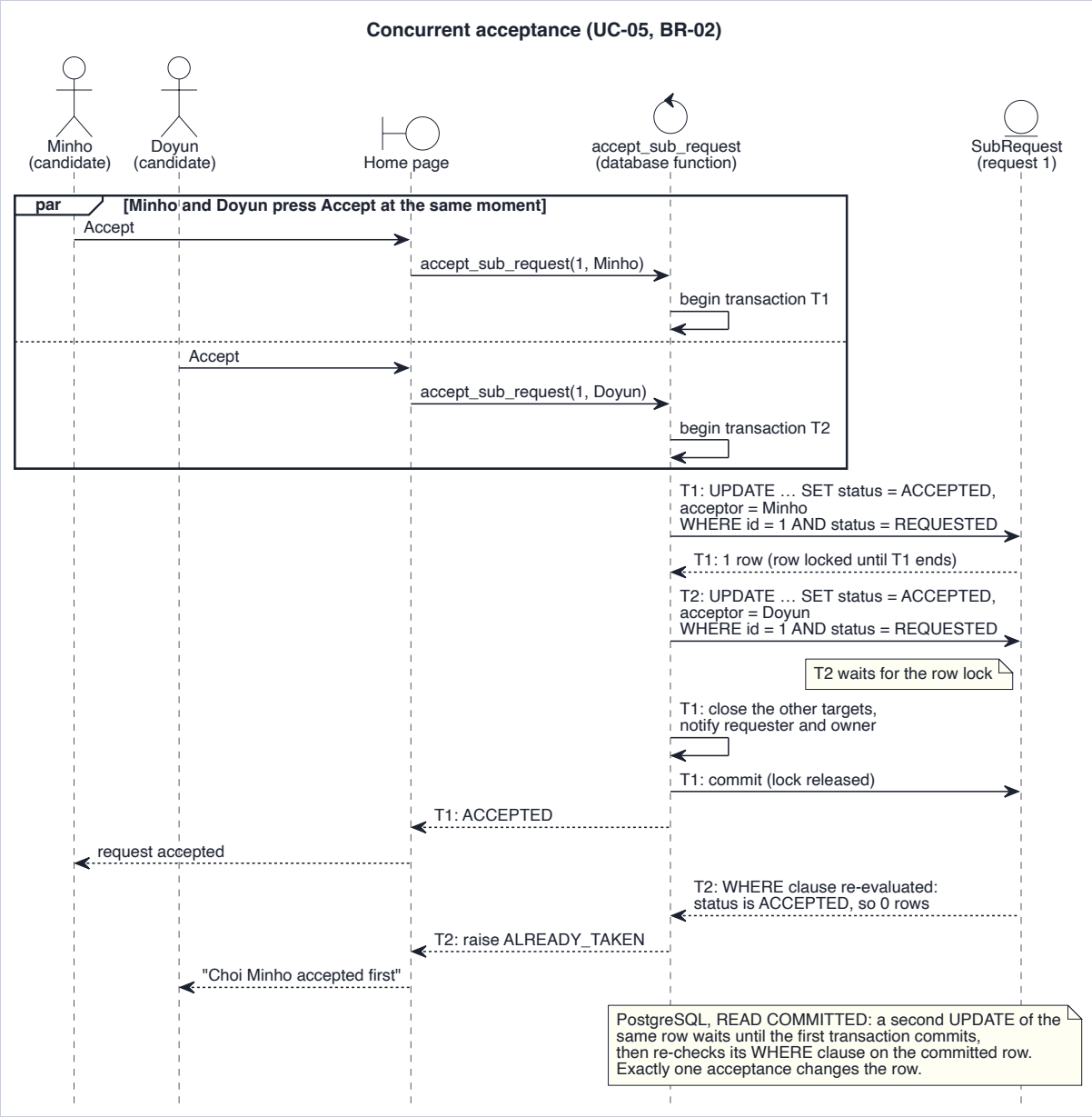


Figure 11. Concurrent acceptance of one request (UC-05, BR-02)

Figure 11 shows the case that the proposal feedback asked about: two candidates press *Accept* on the same request at the same moment. Each press is one call of the accept function and one transaction. The first call changes the request with its conditional statement, closes the other targets, notifies the requester and the owner, and commits. The second call waits for the row until the first transaction ends; PostgreSQL then evaluates its condition again on the changed row [9], finds the status no longer REQUESTED, changes no row and refuses with “Choi Minho accepted first”. No step runs in the browser between reading and writing the request, so two acceptances can never both succeed (NFR-04).

8. Main user interface screens

The screens below were captured from the system described in Section 2.4: the React front end running against the deployed database, with the demo data of Section 2.5 and the demo clock at Monday 2026-09-28 09:00 unless a caption says otherwise. Each screen calls only the database functions listed in Appendix B. A worker has four pages — *Home, Schedule, Swaps* and *Me* — and the owner five — *Home, Schedule, Staff, Pay* and *Settings*; on a desktop they are listed in a left navigation and on a phone they become a bottom tab bar. The header shows the store, the demo clock, the notification bell with its unread count and the signed-in user. Colour is used sparingly: blue marks the signed-in worker's own shifts and the single primary button of each page, and amber is reserved for substitution (a dashed outline for an open request, an amber edge and stripes for a handed-over shift). Status chips carry text, not only colour. The screens are shown in English; the interface is in Korean by default (Figure 25).

The earlier prototype, whose screens the proposal and the team's review in Section 1.2 used, kept its data in the browser; it is replaced by this system and is not shown here.

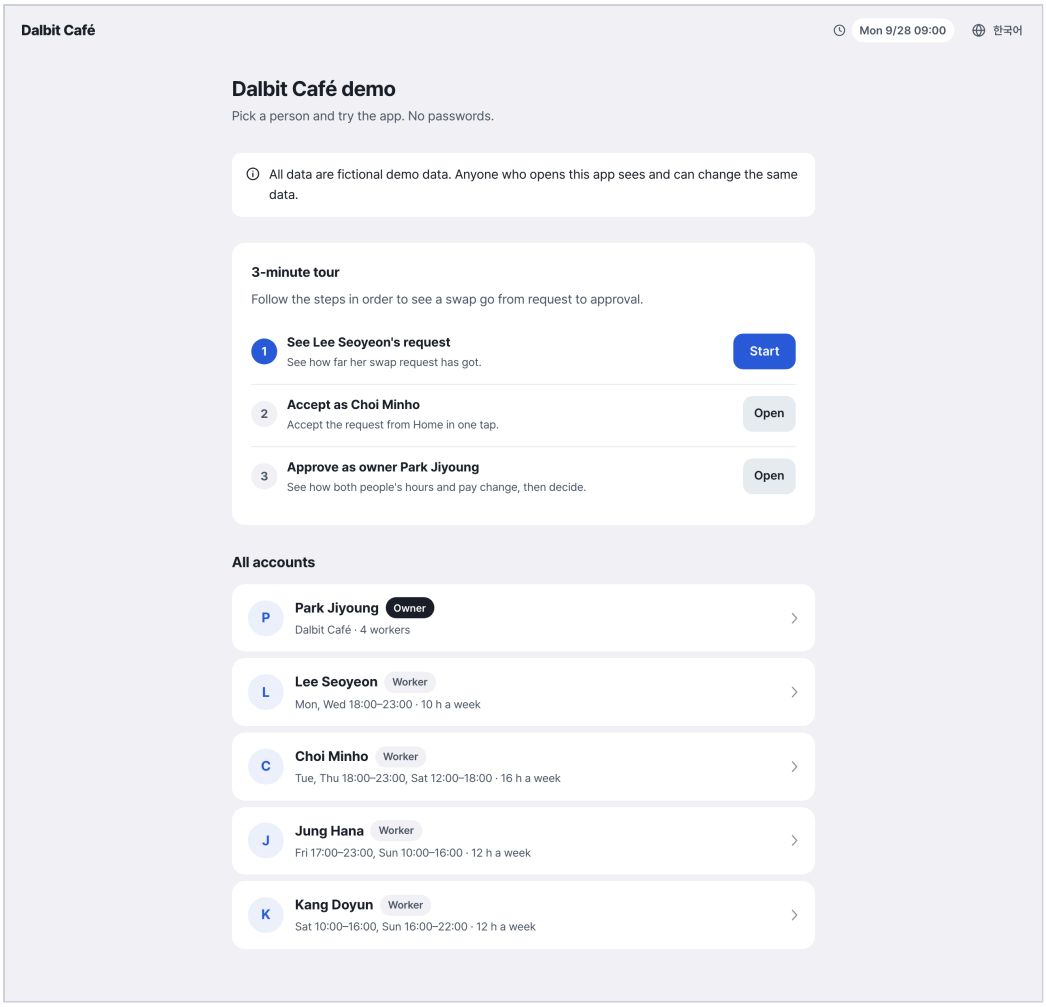


Figure 12. Sign-in (UC-13, FR-20). The notice states that all data are fictional demo data shared by everyone who opens the app; the three-step tour leads from Lee Seoyeon's request to Choi Minho's acceptance and the owner's approval; below, all accounts: the owner (with the number of workers) and the four workers with their weekly hours. There are no passwords.

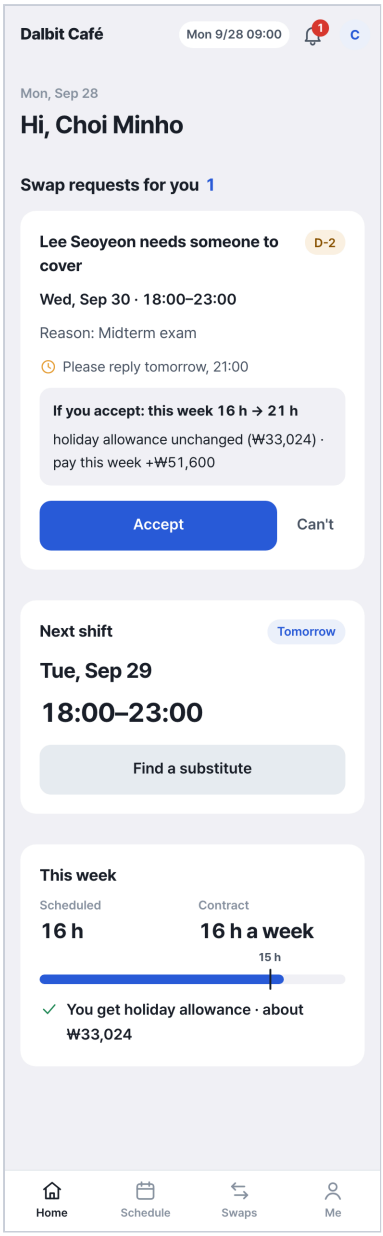


Figure 13. Home of Choi Minho on a phone (UC-05, NFR-01, NFR-13). The card under "Swap requests for you" shows Seoyeon's Wednesday shift, the reason, the reply deadline and the effect on his week (16 h → 21 h, holiday allowance unchanged), with *Accept* as the page's only primary button and *Can't*.

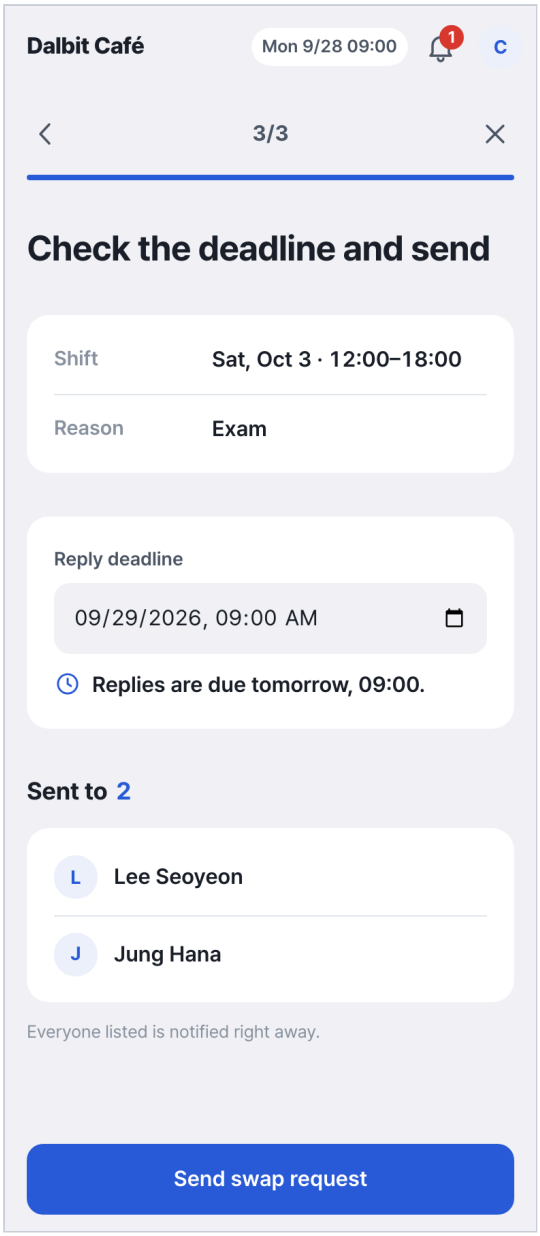


Figure 14. Request flow, step 3 of 3 (UC-04, BR-01, BR-03), Choi Minho for Saturday 10/3 12:00–18:00: the reply deadline pre-filled with the earlier of now + 24 h and the shift start, and the co-workers who will be asked — Lee Seoyeon and Jung Hana; Kang Doyun is not asked because his Saturday shift overlaps.

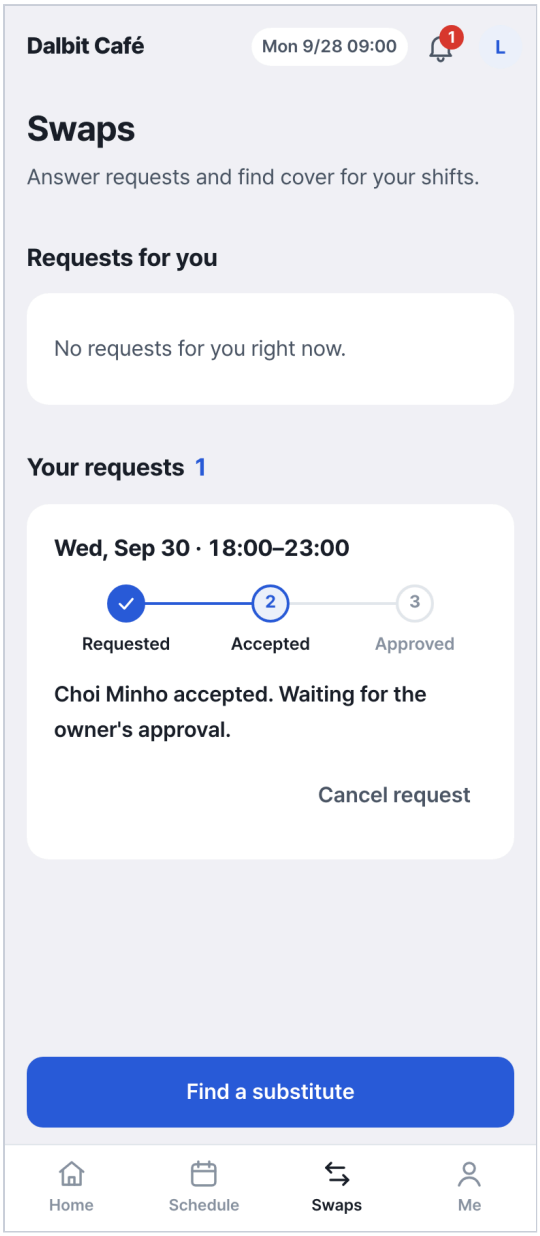


Figure 15. Swaps of Lee Seoyeon after Minho accepted (UC-05, UC-12): the progress Requested → Accepted → Approved at step 2, "waiting for the owner's approval", and *Cancel request* while the request is open.

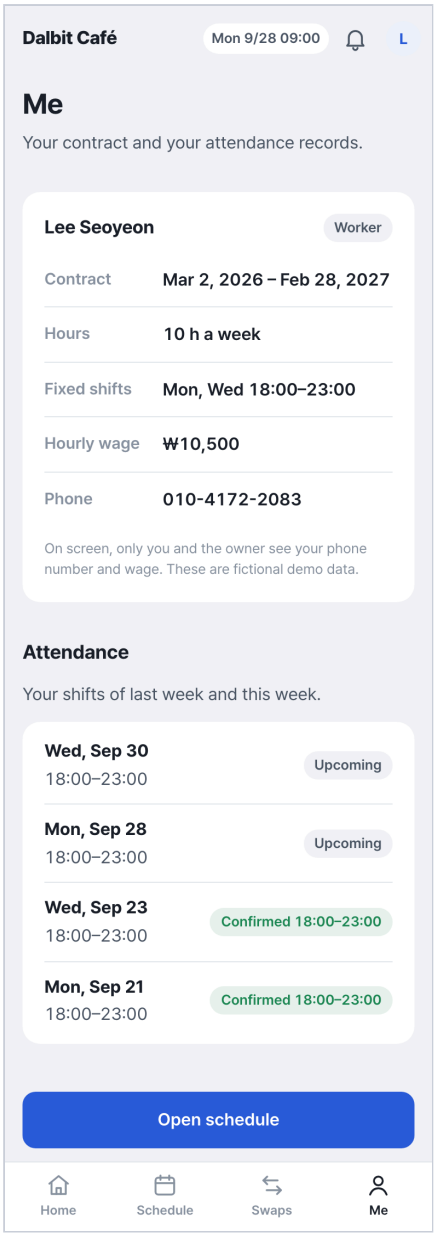


Figure 16. Me of Lee Seoyeon (UC-07, NFR-11): contract, fixed shifts, hourly wage and phone with the note that, on screen, only she and the owner see them and that the data are fictional, and her attendance of last week and this week; a shift can be recorded only after it has ended (BR-17).

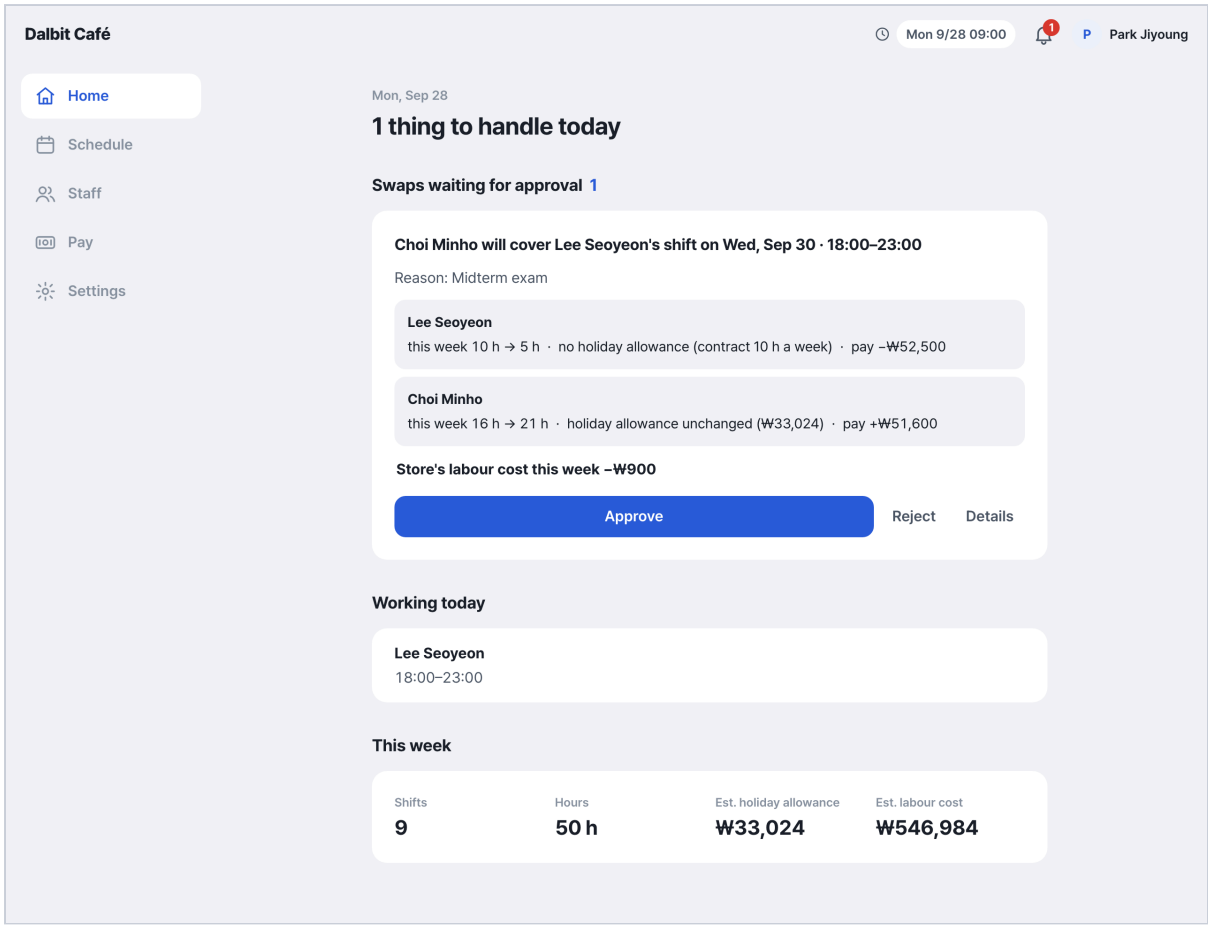


Figure 17. Home of the owner with the decision of S3 (UC-06, FR-11, BR-16). One line per worker: Seoyeon this week 10 h → 5 h, no holiday allowance (contract 10 h a week), pay −KRW 52,500; Minho 16 h → 21 h, holiday allowance unchanged (KRW 33,024), pay +KRW 51,600; the store's labour cost this week −KRW 900. *Approve*, *Reject* and *Details*.

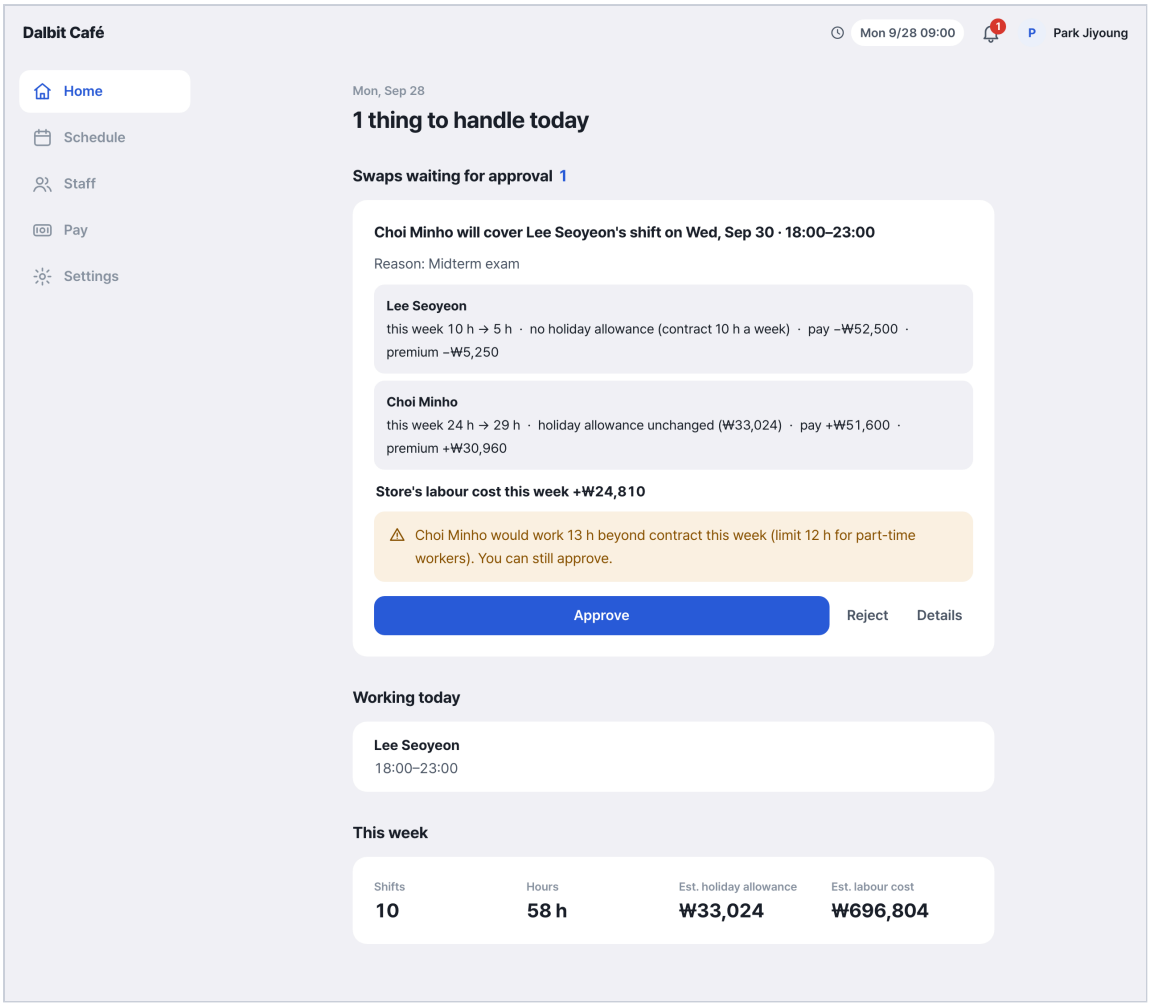


Figure 18. The same decision in a workplace with five regular employees and Minho's extra Friday shift (S8; FR-24, BR-15, BR-16): Minho 24 h → 29 h with premium +KRW 30,960, Seoyeon premium −KRW 5,250, the store's labour cost +KRW 24,810, and the warning that Minho would work 13 hours beyond his contract, above the limit of 12 for part-time workers; *Approve* stays enabled.

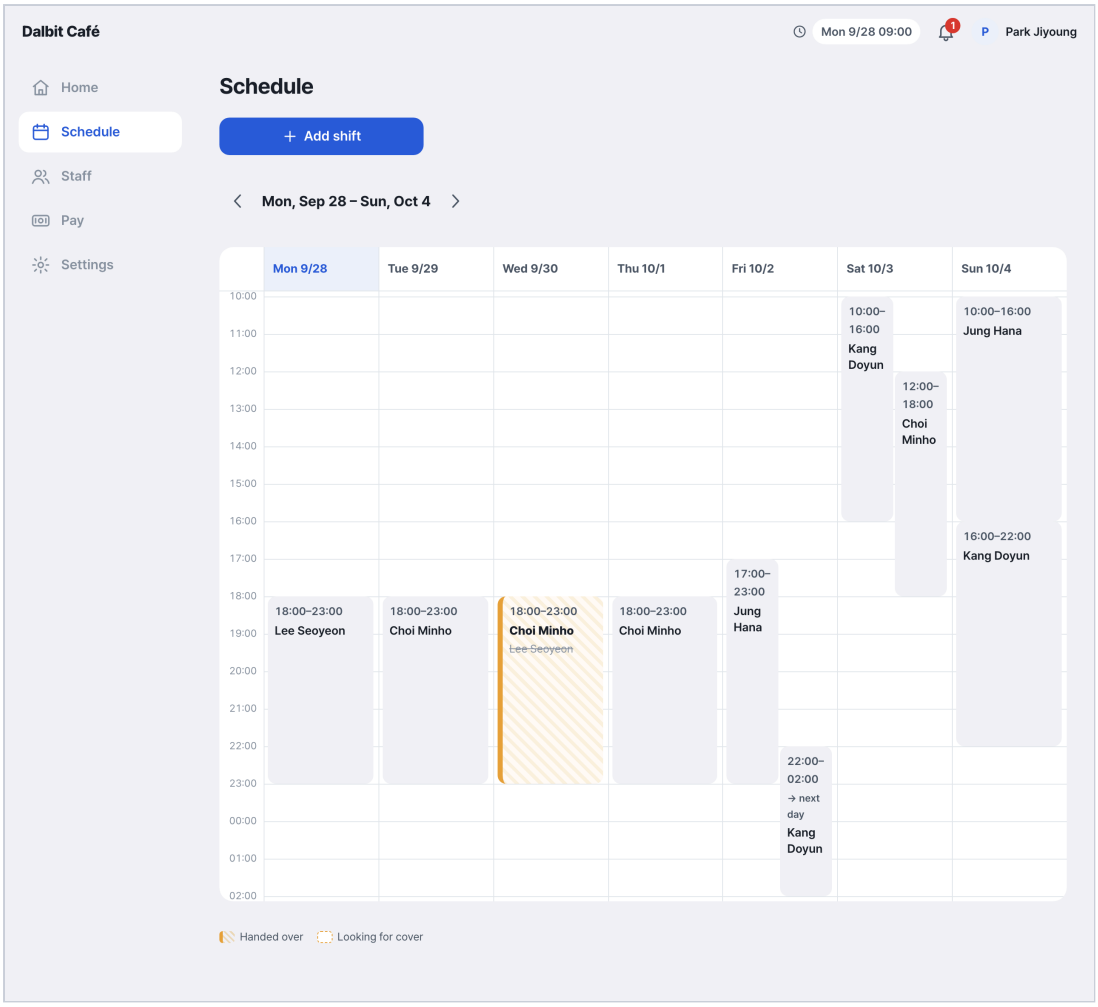


Figure 19. Schedule, owner view, after the approval of S3 and with the shift of S9 (UC-03, BR-04, BR-05). Wednesday 9/30 18:00–23:00 shows Choi Minho with Lee Seoyeon struck through on an amber, striped background; Kang Doyun's Friday 22:00–02:00 shift is drawn on Friday and marked "→ next day". *Add shift* is the primary action.

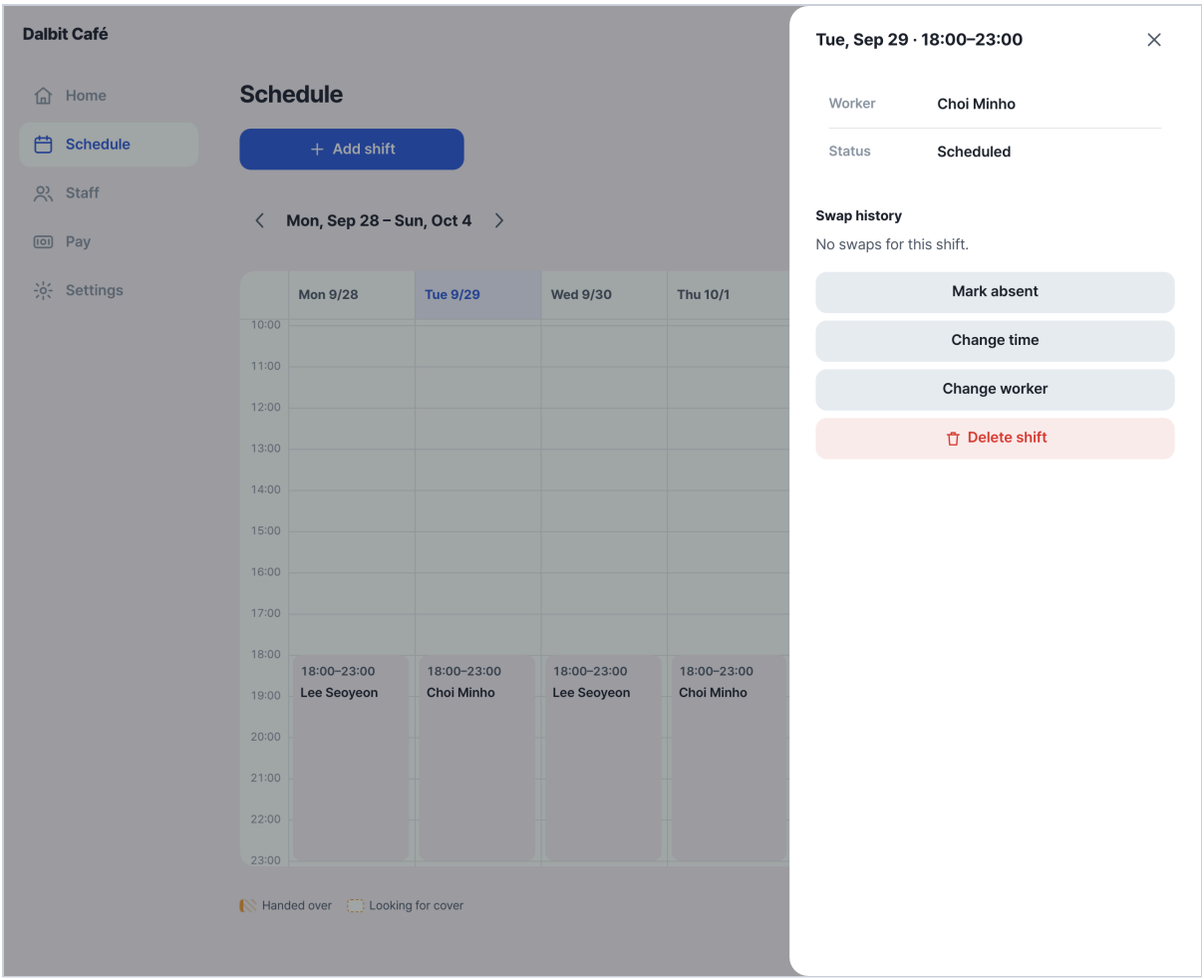


Figure 20. Shift sheet of Choi Minho's Tuesday 9/29 18:00–23:00 shift, with the demo clock at 23:30 (S11; UC-03, FR-25): the shift has started, is still scheduled and has no attendance record, so the owner can mark it absent; changing and deleting the shift are offered as well.

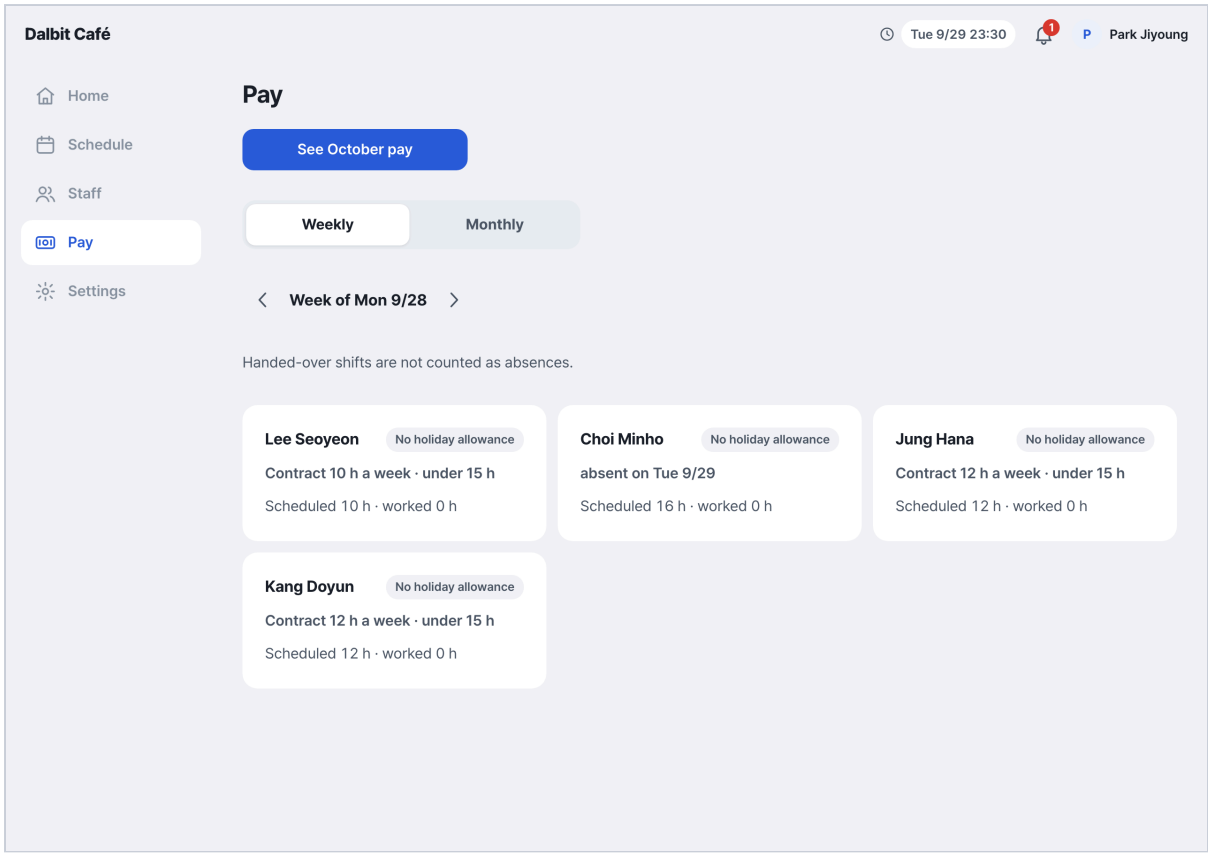


Figure 21. Pay, tab Weekly, week of 9/28 after S11 (UC-08, BR-06, BR-07): Choi Minho has no holiday allowance because of the absence on 9/29; the other workers' contracts are under 15 hours. The note states the store's policy for handed-over shifts.

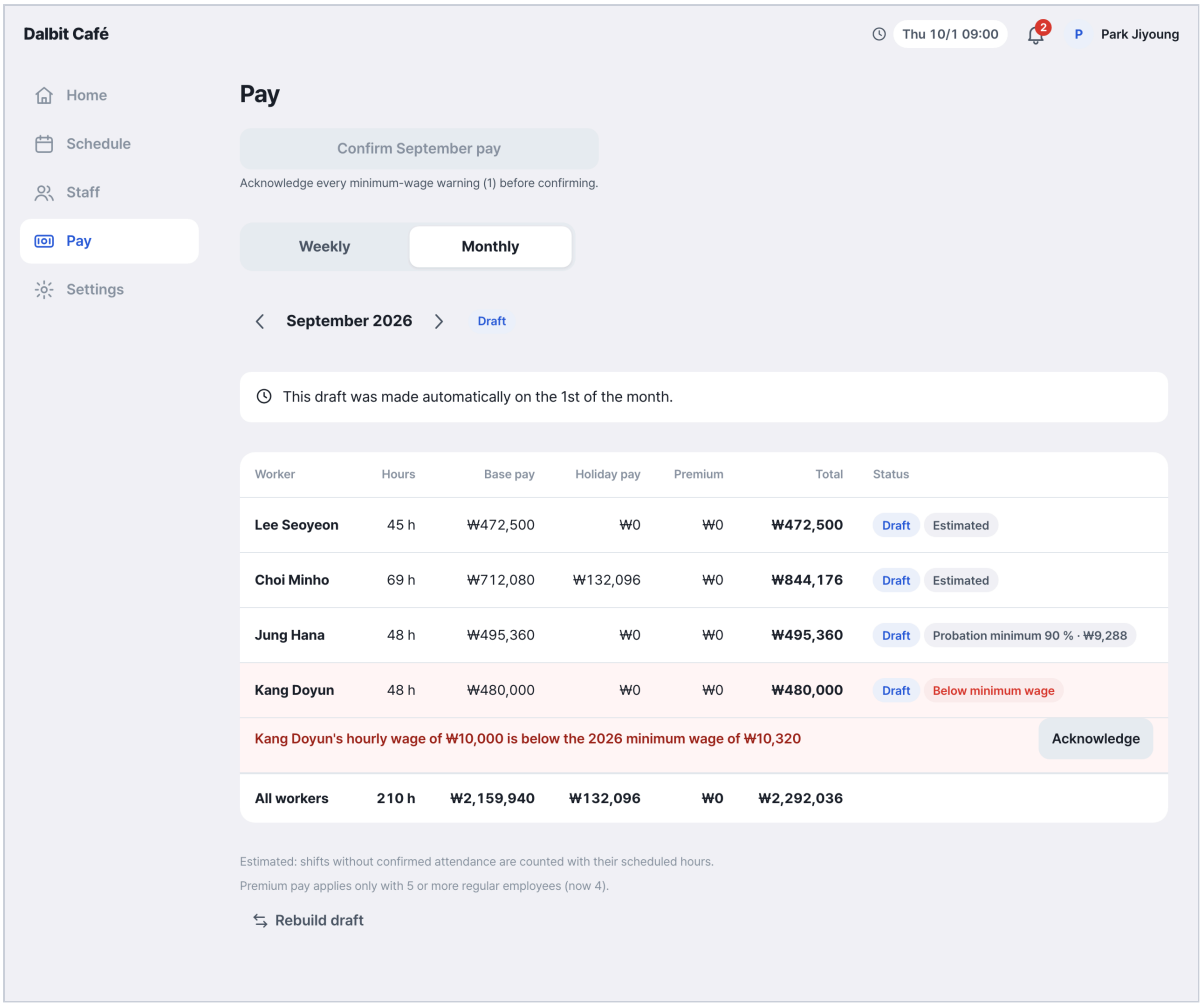


Figure 22. Pay, tab Monthly, September 2026 at 10/1 09:00 (S6, S10; UC-09, UC-14, FR-15, FR-16): the draft was prepared automatically; Seoyeon KRW 472,500 and Minho KRW 844,176 (estimated), Hana KRW 495,360 with the probation minimum KRW 9,288, Doyun KRW 480,000 below the minimum wage with *Acknowledge*; total KRW 2,292,036. *Confirm September pay* is disabled with its reason until the warning is acknowledged.

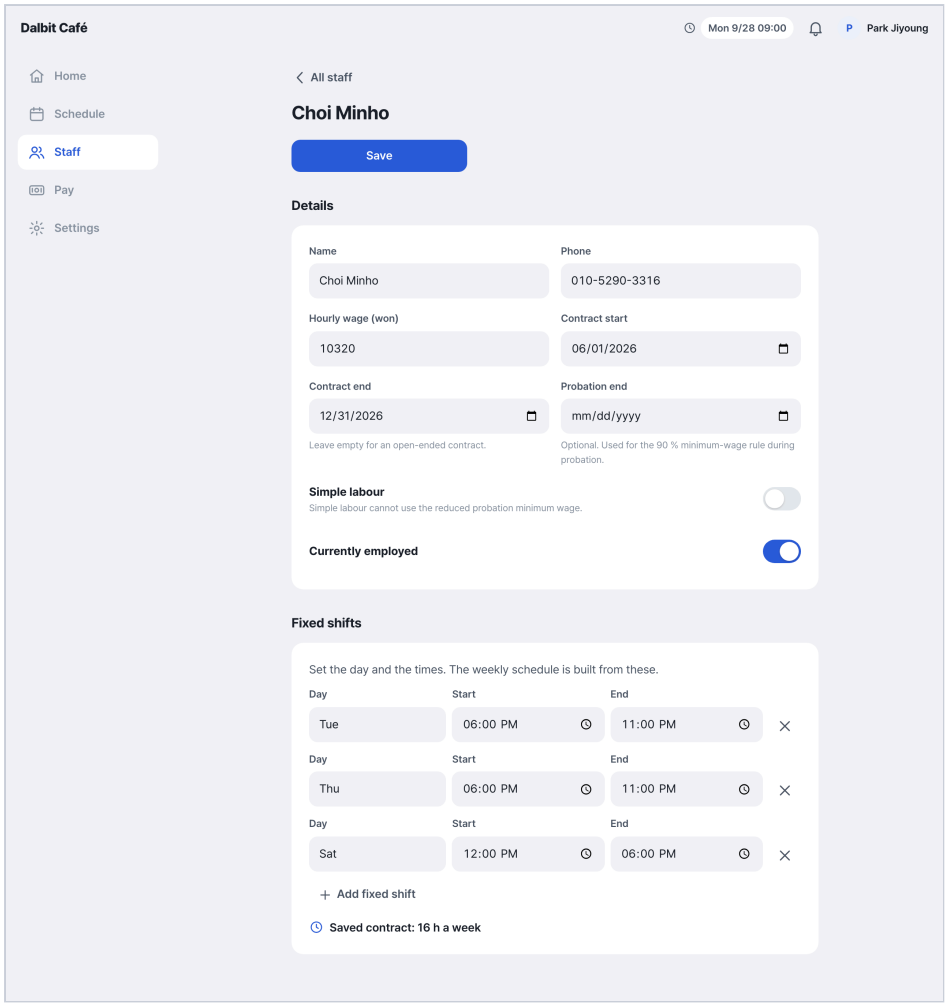


Figure 23. Staff detail of Choi Minho (UC-01, FR-01, FR-02): name, phone, hourly wage, contract dates, probation end, simple labour and “currently employed”, and the fixed shifts Tuesday and Thursday 18:00–23:00 and Saturday 12:00–18:00 with the resulting 16 contractual hours a week.

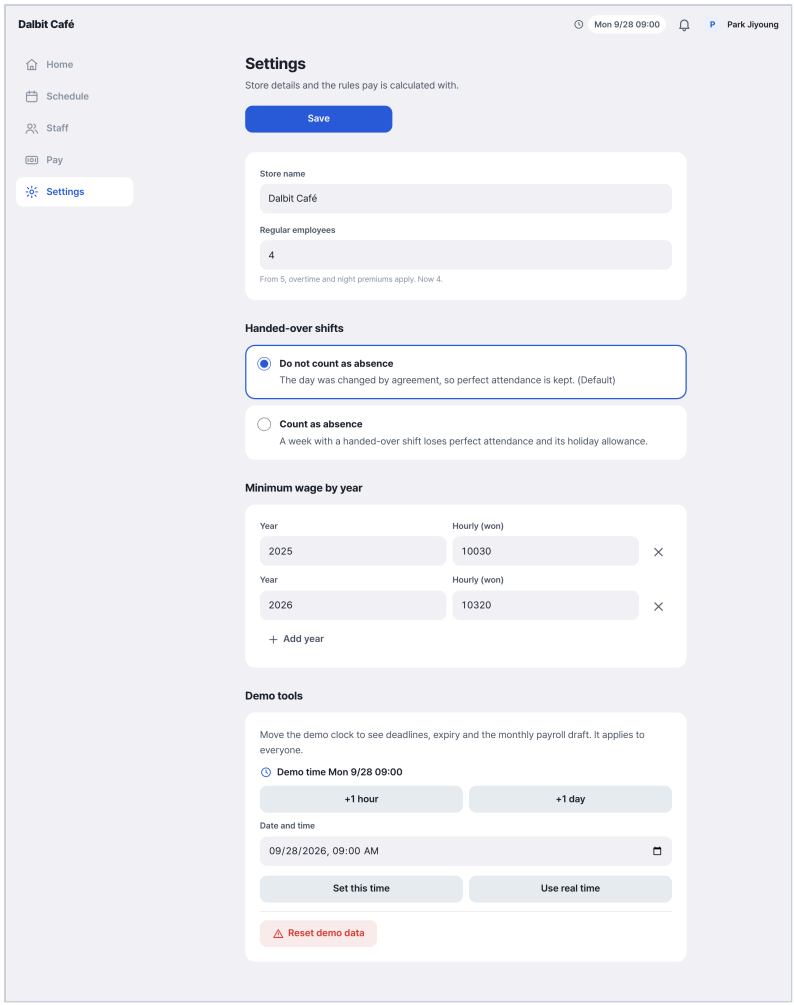


Figure 24. Settings (UC-10, FR-17, FR-20): store name, number of regular employees (premiums from five), the policy for handed-over shifts (“Do not count as absence”, the default, or “Count as absence”), the minimum wage by year, and the Demo tools that move the demo clock or restore the demo data.



Figure 25. The owner's Home of Figure 17 in Korean on a 390 px phone (NFR-02, NFR-13): the same decision card with the same numbers, above the bottom tab bar of the owner's five pages.

Table 41 maps every use case to the screen that realises it. UC-11 and UC-14 have no screen of their own: the System Clock function runs whenever a screen is loaded. Notifications (FR-21) are reached from the bell in the header of every screen.

Table 41. Use cases and screens

Screen	Role	Use cases	Figure
Sign-in (demo)	All	UC-13	12

Screen	Role	Use cases	Figure
Home	Worker	UC-05, UC-04 (entry)	13
Swaps and the request flow	Worker	UC-04, UC-05, UC-12	14, 15
Me	Worker	UC-07 (recording)	16
Home	Owner	UC-06, UC-07 (confirmation), UC-14 (notice)	17, 18, 25
Schedule and the shift sheet	Owner, Worker	UC-03, UC-04 (entry), UC-06 (result)	19, 20
Pay, Weekly	Owner	UC-08	21
Pay, Monthly	Owner	UC-09, UC-14	22
Staff	Owner	UC-01	23
Settings and Demo tools	Owner	UC-10, UC-13 (demo clock, reset)	24
(no screen; runs on every screen load)	System Clock	UC-11, UC-14	–

9. Requirements validation

Requirements validation checks that the requirements define the system that the customer really wants (Chapter 8). We check the five properties of Chapter 8 with the three techniques it names: requirements reviews, prototyping and test-case generation.

9.1 Requirements review

Two kinds of review are used. In the internal review the requirements lead's FR, NFR and business-rule tables are reviewed by the other two members against the problem statement and the scenarios; the documents are reviewed in a shared document tool that keeps every version with its author and highlights what changed between versions, so that each comment can be followed to the requirement it changed. In the review with the owner of the reference store (planned for Week 8), the use cases will be read out as scenarios in the owner's own terms and the owner will be asked whether each step matches what is needed; the owner's answer to interview question 6 will decide the default of the BR-06 policy, and the system keeps both options in any case.

Table 42. Validation checklist (Chapter 8)

Property	Question	How we check it

Property	Question	How we check it
Validity	Does the system provide the functions that best support the owner's and workers' needs?	Each problem P1–P3 is traced to functional requirements (P1 → FR-06 to FR-10, FR-22, FR-21; P2 → FR-11, FR-12, FR-14; P3 → FR-11, FR-14, FR-15, FR-16, FR-24); review with the owner.
Consistency	Are there conflicting requirements?	One specification is the single source of all identifiers; the state diagrams are checked against the scenarios (Section 7.3); the dependency check of Table 9 found and removed two conflicts between *Must* and *Should*.
Completeness	Are all required functions included?	Every FR maps to a use case, every use case to a page and a database function (Table 41, Appendix B), and every business rule to a test group (Appendix A).
Realism	Can the requirements be implemented with the available time and technology?	MoSCoW priorities, with increment 1 independent of increment 2; a managed database on a free plan needs no paid hosting (NFR-16); the database functions of all use cases already run against the test cases.
Verifiability	Can each requirement be checked?	Each NFR has a verification method (Table 24); each FR has an observable outcome; the business rules have numeric boundaries (15 h, 40 h, 5 employees, 12 h, 90 %, three months, 365 days, 30 minutes and 16 hours).

9.2 Prototyping

The screens of Section 8 are used to validate the requirements in two ways. The team walks through scenarios S1 to S11 and records every step where a screen does not match the use-case description; this walk-through is also the acceptance checklist of the final prompt. The owner and the two co-workers will each be asked to perform one task without help: the owner approves the request of S3 and reads the cost line of the approval card, one co-worker creates the request of S1, and the other accepts it from *Home* as in S2. The team's own walk-throughs of the prototype already changed three requirements (Section 1.2); the sessions with the owner and the co-workers are planned for Week 10.

9.3 Test-case generation

Chapter 8 recommends writing tests for requirements to find requirements that are ambiguous or cannot be tested, and Chapter 10 recommends tests on and just off each boundary. We derived one test group per business rule before implementing the user interface. The groups

make the boundaries of the requirements explicit: 15 contractual hours are eligible and 14.99 are not; holiday hours are capped at 40 contractual hours; premiums start at five regular employees; 12 hours beyond contract give no warning and 13 do; a shift may last from 30 minutes to 16 hours and may end after midnight. Table 43 lists the groups and Appendix A the individual test cases with their inputs and expected results; they are fixed in Week 8, before the final prompt.

Table 43. Business rules and test groups

Business rule	Test group	What the group checks	Related NFR
BR-01, BR-05	TC-01x	Candidate eligibility and overlap on the time line, also across midnight and contract ends	–
BR-03	TC-02x	Request validity (own shift, future, one open request, deadline)	–
BR-02	TC-05x	First acceptance: a second acceptance is refused, also when concurrent	NFR-04
BR-04	TC-06x	Approval effect on the shift, the request and both weeks	–
BR-06, BR-07	TC-08x	Holiday allowance at 14.99 h and 15 h, the 40 h cap, both policies, unmarked and absent shifts (S3, S11)	NFR-10
BR-08	TC-09x	Premium at four and five employees, night hours across midnight, hours beyond contract (S8)	NFR-10
BR-09, BR-10	TC-10x	Minimum-wage warning and each probation condition (S6)	NFR-10
BR-11	TC-11x	Monthly payroll totals and rounding in minutes	–
BR-12	TC-12x	Expiry when a screen is loaded and inside accept and decide (S4)	–
BR-13	TC-13x	No taker at creation and after the last “can't” (S5)	–
BR-14	TC-14x	Automatic drafts for every missing past month (S10)	–
BR-15	TC-15x	Warning at 13 h, none at 12 h, none with four employees (S8)	NFR-10
–	TC-16x	The API roles cannot write tables; functions check their arguments	NFR-14
BR-16	TC-17x	Swap cost estimate (S3, S7, S8)	NFR-10
BR-17	TC-18x	Attendance only after the end, plausible times, clock-out after midnight	–
–	TC-19x	Seed and reset of the demo data	NFR-05

The test cases run automatically against the database script in an embedded PostgreSQL (PGlite); at the time of this report all 84 pass. The first-acceptance case is also run with two concurrent connections against the deployed PostgreSQL database (record of 10 October 2026 kept with the tests): one acceptance succeeds and the other, answered only after the first transaction has committed, is refused with “Choi Minho accepted first”.

10. Progress, risks and next steps

Work package 3 of the WBS (requirements elicitation and specification) is complete with this report except the interviews (3.1.1), which are planned for Week 8, and the analysis models of work package 4.1 are drawn. In response to the proposal feedback the team also moved two items forward: the test cases (work package 6.1.1, planned for Week 12) are written and fixed before implementation of the user interface, and the database functions, which carry every business rule, are implemented and tested against them. The user interface of all thirteen use cases has also been built from the same specification with Claude Code (Section 8) and is checked automatically against the deployed database: the walk-through of the scenarios S1–S11 with their numbers and the layout audit of every page at 360 and 1280 px in Korean and English pass (145 checks; record of 10 October 2026 kept with the tests). This first complete build is used to validate the requirements; in Weeks 10 and 11 the increments are built again from the final prompt, to verify that the prompt reproduces them (goal G6 of the proposal: a system that the final prompt reproduces).

Table 44. Plan to the final report

Week	Dates	Work	Result
8	10-19 to 10-25	Interviews with the owner and two co-workers (Section 3); design models (design class diagram, sequence diagrams of UC-04, UC-05, UC-06 and UC-09); test cases of Appendix A fixed	Interview notes; design models; frozen test cases
9	10-26 to 11-01	Final prompt for Lovable, first version (database script, design system, pages, checklist); first trial of its first step (the database script) in Lovable	Prompt, first version; trial record with the credits used
10	11-02 to 11-08	Increment 1 (*Must* use cases) built again from the final prompt in an empty project with Claude Code and checked against Appendix A; sessions with the owner and the co-workers on the system (Section 9.2)	Increment 1 by 11-06; session notes
11	11-09 to 11-15	Increment 2 (*Should* use cases) built again from the final prompt; changes from the sessions	Increment 2 by 11-13
12	11-16 to 11-22	Deployment; all test groups and the scenario walk-through on the deployed system	Deployed URL by 11-20

Week	Dates	Work	Result
13	11-23 to 11-29	The final prompt re-run in an empty Lovable project; every fix recorded with its reason	Final prompt frozen
14	11-30 to 12-06	Final report, slides, presentation	Submission by 12-04

Table 45. Risks

ID	Risk	Response
R1	Legal interpretation where the law leaves the method open: attendance when a shift is handed over (BR-06), and how the hours of a part-time worker beyond contract are counted (BR-08 (iv)) — the labour ministry's examples count them against the contractual hours of the day, the system against those of the week.	The attendance rule is a store setting (BR-06), EXCUSED by default, and both policies are tested (TC-08x); the weekly method of BR-08 (iv) is stated as a design decision and will be reviewed with the owner.
R2	Scope creep.	The Won't list of the proposal is fixed; a new requirement enters the specification only with a team decision and a priority.
R3	Reproducibility of the final prompt in Lovable, whose output varies from run to run and whose free plan grants five build credits a day and at most 30 a month [24].	Every rule is in the database script, which is attached to the first message and applied unchanged; the prompt is a short ordered set of messages with an acceptance checklist; a trial in Week 9 and a full re-run in Week 13; the credits used are recorded.
R4	Pay rules implemented incorrectly.	Rules in database functions with the boundary tests of Appendix A.
R5	Reduced availability of members during examinations.	Each work package has an owner and a reviewer who can take over.
R6	A free-plan database is paused after inactivity [21].	A scheduled read every six hours, a weekly check of the deployed URL, and a check before every submission (NFR-16).

ID	Risk	Response
R7	Notifications are in-app only, so a candidate sees a request only when opening the application; the quick answers that goals G1 and G2 of the proposal aim at (no telephone round of the owner; an answer in two clicks) depend on it.	Unread count in the header; the reply deadline on every request card; screens refreshed every 10 seconds and when the window regains focus (NFR-15); the default deadline of 24 hours leaves time to answer. Messaging services stay out of scope (Won't list).

Appendix A. Test cases

The 84 test cases below are written against the business rules and the demo data of Section 2.5. Each has an identifier of its test group (Table 43), the situation with its input and expected result, and the rules it checks; values on and just off each boundary are included. Every case is an automated test of the database functions; the scenarios S1–S11 appear here with the numbers of Section 6. The group numbers follow the test plan of the proposal, in which the numbers 03, 04 and 07 are not used.

Table 46. Test cases of the database functions

TC	Input and expected result	Rules / requirements
TC-01x Eligibility, overlap on the time line, weekly schedule (BR-01, BR-05, FR-02, FR-04, FR-05, FR-25)		
TC-011	S9: Fri 2026-10-02 22:00–02:00 belongs to Friday and the week of 09-28, lasts 4 h; Sat 01:00–05:00 overlaps it	BR-05
TC-012	single shifts: Mon 22:00–02:00 overlaps Tue 01:00–05:00, Sun 22:00–02:00 overlaps the next Monday 01:00–05:00	BR-05, FR-05
TC-013	fixed slots of one worker are compared on the weekly time line, across midnight and from Sunday into Monday	FR-02, BR-05
TC-014	S1: the seed request asks Minho, Hana and Doyun, each with a REQUEST_RECEIVED notification	BR-01, FR-07
TC-015	S7 / S5: Doyun is not asked for Minho's Sat 12:00–18:00 (his Sat 10:00–16:00 overlaps); a shift that ends exactly when the requested shift starts does not exclude	BR-01
TC-016	a co-worker whose overnight shift of the previous evening runs into the shift is not eligible	BR-01 on the time line
TC-017	only active workers within their contract period are asked: Doyun after 2026-11-15, an inactive Hana	BR-01, FR-04
TC-018	generate_week creates each fixed slot once and only within each contract; Doyun has no shift after 2026-11-15	FR-04
TC-019	generate_week skips a slot whose shift was moved or that would overlap a shift the worker already has	FR-04
TC-01A	add, move and delete a single shift; SHIFT_LOCKED, SHIFT_HAS_HISTORY, OUTSIDE_CONTRACT, WORKER_INVALID, TIME_INVALID	FR-05
TC-01B	FR-25: the owner marks a started SCHEDULED shift absent with or without a record; not before the start, not twice	FR-25
TC-01C	week board and shift detail: names, open-request and handover markers, allowed actions	UC-03
TC-02x Request validity (BR-03)		
TC-021	a valid request: own future SCHEDULED shift, deadline after now and not after the start; one target and notification per candidate	BR-03, FR-06, FR-07

TC	Input and expected result	Rules / requirements
TC–022	invalid requests are refused: SHIFT_NOT_FOUND, NOT_YOUR_SHIFT, NOT_SCHEDULED, OPEN_REQUEST_EXISTS, DEADLINE_INVALID, SHIFT_STARTED	BR-03
TC–023	at most one open request per shift; a closed request does not block a new one	BR-03
TC-05x First acceptance (BR-02, NFR-04)		
TC–051	S2: the first acceptance wins; Doyun's later acceptance is refused with ALREADY_TAKEN naming Choi Minho	BR-02, FR-09, FR-10
TC–052	accepting a FAILED or an EXPIRED request is refused with REQUEST_CLOSED	BR-02, BR-13
TC–053	ACCEPTOR_BUSY: a candidate with an overlapping shift cannot accept; the overlap is checked again at approval	BR-02
TC–054	only a target can answer: NOT_A_TARGET for the requester and the owner, REQUEST_NOT_FOUND for an unknown request	BR-02
TC–055	a single "can't" keeps the request open and notifies no one; that candidate cannot accept afterwards	FR-22
TC-06x Approval effect (BR-04)		
TC–061	approval hands the shift to the acceptor, keeps the original worker, recomputes both weeks and notifies both	BR-04, FR-12
TC–062	rejection leaves the shift unchanged; only an ACCEPTED request can be decided; the decision is APPROVE or REJECT	BR-04
TC–063	a shift swapped twice keeps its first worker as original worker	BR-04
TC-08x Holiday allowance (BR-06, BR-07)		
TC–081	holiday allowance needs 15 contractual hours: 14.99 h → none, 15 h → 3 h	BR-06 a, BR-07
TC–082	holiday hours are capped at 40 contractual hours: 16 h → 3.2 h, 45 h → 8 h	BR-07
TC–083	both substitution-attendance policies: a shift given away by an approved swap keeps (EXCUSED) or breaks (ABSENT) attendance	BR-06
TC–084	S3 holiday allowance: Minho 3.2 h = 33,024 before and after taking the shift; Seoyeon none (contract 10 h)	BR-06, BR-07

TC	Input and expected result	Rules / requirements
TC–085	S11 no-show: at 2026-09-29 23:30 the owner marks Minho's Tue shift absent without a record; his week loses 33,024 → 0	FR-25, BR-06
TC–086	BR-06 default: a past shift that was not marked absent counts as worked (no attendance needed)	BR-06
TC–087	weeks of a month whose payroll is confirmed are not recomputed	FR-14
TC–088	monthly holiday pay is the sum over the weeks whose Sunday falls in the month	BR-07, BR-11
TC-09x Premium pay (BR-08)		
TC–091	premium is zero below five regular employees and paid from five	BR-08
TC–092	night hours are the minutes between 22:00 and 06:00 on the actual time line: 22:00–02:00 is 4 h, on its work date	BR-08 iii, S9
TC–093	premium categories: daily beyond 8 h, weekly beyond 40 h, night hours, part-time beyond contract counted once	BR-08 i-iv
TC–094	in a monthly payroll the days of an ISO week outside the month only decide where the first C and the first 40 hours end	BR-08, monthly payroll
TC–095	S8 worked: October pays Minho 13 h beyond contract = 67,080 plus 1 night hour; September only night hours	BR-08 iv
TC-10x Minimum wage and probation (BR-09, BR-10)		
TC–101	S6: September draft at 2026-10-01: Doyun's warning, Hana's probation minimum 9,288; confirmation waits for the acknowledgement	BR-09, BR-10, FR-16
TC–102	probation applies only when all BR-10 conditions hold (one-year contract incl. 2026-09-01 to 2027-08-31, first 3 months, until probation_end, not simple labour)	BR-10
TC–103	the probation floor is exactly 9,288; a regenerated draft needs a new acknowledgement	BR-09
TC-11x Monthly payroll (BR-11)		
TC–111	S6 September draft: Seoyeon 472,500, Minho 844,176 (64 h worked + 5 h scheduled, four holiday weeks), Hana 495,360, Doyun 480,000 with warning, total 2,292,036	BR-11
TC–112	hours are summed in minutes and converted once; WORKED shifts count their recorded minutes	BR-11

TC	Input and expected result	Rules / requirements
TC–113	ABSENT shifts are paid 0; a month of worked shifts only is not estimated	BR-11
TC–114	a confirmed month is read-only; NO_DRAFT and MONTH_INVALID	FR-15
TC-12x Expiry (BR-12)		
TC–121	S4: nobody answers; at 2026-09-29 21:00 the System Clock expires the request and notifies requester and owner	BR-12, FR-18
TC–122	an ACCEPTED request expires at the shift start; expire_overdue_requests returns the number expired	BR-12
TC–123	accept_sub_request evaluates the expiry first: a late acceptance is refused even if no screen was loaded	BR-12
TC–124	decide_sub_request evaluates the expiry first: approval at the deadline or after the shift start is refused	BR-12, BR-04
TC–125	cancel: the requester cancels; every PENDING or ACCEPTED target is told; a closed request cannot be cancelled	UC-12, FR-19
TC-13x No taker (BR-13)		
TC–131	S5: Hana's Sun 10-04 request; all three answer "can't" → FAILED; Hana and the owner receive REQUEST_FAILED and acknowledge it	BR-13, FR-22
TC–132	no eligible worker at creation: the request ends FAILED at once without targets	BR-13, FR-08
TC-14x Automatic payroll drafts (BR-14)		
TC–141	S10: no draft is due before 2026-10-01; at 10-01 09:00 the System Clock drafts September once and tells the owner	BR-14, FR-23
TC–142	every missing month is drafted: clock 2026-09-28 → 2026-12-02 drafts September, October and November, one notification each	BR-14
TC-15x Part-time weekly limit warning (BR-15)		
TC–151	S8: with 5 regular employees the preview shows Minho 24 → 29 h and warns of 13 h beyond contract (limit 12); approval is not blocked	BR-15, FR-24
TC–152	S8 with Friday 10:00–17:00: Minho 23 → 28 h is exactly 12 h beyond contract, no warning	BR-15
TC–153	S8 with 4 regular employees: 13 h beyond contract but no warning and no premium	BR-15, BR-08

TC	Input and expected result	Rules / requirements
TC-16x Security of the database (NFR-14)		
TC–161	the API roles read every table but cannot insert, update, delete or truncate any of the 11 tables	NFR-14
TC–162	the API roles change data only through the functions: S2 runs as anon; the internal helper functions cannot be called	NFR-14
TC–163	functions validate their arguments, also when called by the API role	NFR-14
TC–164	row-level security on all 11 tables with read policies for anon and authenticated; execute grants only on the API	NFR-14
TC-17x Swap cost estimate (BR-16)		
TC–171	S3: Seoyeon 10 → 5 h, pay –52,500; Minho 16 → 21 h, holiday 33,024 unchanged, pay +51,600; premium 0; store –900	BR-16, FR-11
TC–172	S7: policy ABSENT; Minho gives away Sat 12:00–18:00 to Hana: Minho eligible → not, holiday –33,024, pay –61,920; Hana +61,920; store –33,024	BR-16
TC–173	S8: Minho base +51,600 and premium +30,960 (beyond contract 8 → 13 h, night 2 → 3 h); Seoyeon base –52,500 and premium –5,250 (night 2 → 1 h)	BR-16, BR-08
TC-18x Attendance (BR-17, FR-13)		
TC–181	attendance can be recorded only after the scheduled end (SHIFT_NOT_ENDED), by the shift's worker	BR-17, FR-13
TC–182	23:00 → 18:00 on an 18:00–23:00 shift lies on the next day and is implausible	BR-17
TC–183	an overnight shift: clock-out after midnight is on the next day and accepted; an early clock-in stays on the work date	BR-17
TC–184	the owner confirms a record (WORKED) or the shift stays open; a WORKED or ABSENT shift cannot change again	FR-13, Figure 7
TC–185	the seed history and the week of 09-21 are recorded at the scheduled times and confirmed	Section 2.5, BR-17
TC-19x Seed, reset and the read functions of the screens (NFR-05, Table 48)		
TC–191	the seed reproduces Section 2.5: store, demo clock, minimum wages, accounts and fixed shifts	Section 2.5
TC–192	the seed work history: every fixed slot from contract start to 2026-09-20 WORKED and confirmed, weeks 09-21 and 09-28 generated	Section 2.5

TC	Input and expected result	Rules / requirements
TC-193	the seed payroll: March–August generated, acknowledged and confirmed by the functions; no September row	Section 2.5, BR-14
TC-194	the seed request: Lee Seoyeon, Wed 2026-09-30 18:00–23:00, deadline 09-29 21:00, "Midterm exam", created 08:30	Section 2.5, S1
TC-195	reset_demo_data restores the same data with the same ids, also after changes, and resets the demo clock	NFR-05
TC-196	the SQL file can be run again on the same database (with the API roles): same objects, same demo data	R3
TC-197	the API of Table 48 exist with the exact names, arguments and result types; every function has an empty search_path	NFR-14
TC-19A	demo_accounts: the active accounts, owner first, with weekly contractual hours and fixed shifts	FR-20
TC-19B	get_settings: store, regular employees, policy, demo clock, now and the minimum wages	UC-10
TC-19C	list_workers and worker_detail: the staff with contract data and contractual hours	UC-01
TC-19D	requestable_shifts: own future SCHEDULED shifts without an open request, with the default deadline	UC-04
TC-19E	my_requests: my own requests newest first and the requests for me, open ones first, with the effect on my week	UC-04, UC-05, UC-12
TC-19F	attendance_list: my shifts of the previous and the current week with their records; can_record after the end	UC-07, BR-17
TC-19G	payroll_month and payroll_months: stored rows, flags, status and totals; nothing is generated	UC-09, UC-14
TC-19H	notifications and mark_notifications_read: the inbox newest first with the unread count	FR-21
TC-19I	worker_home: requests for me with the effect on my week, my open requests, next shift, this week	NFR-13, UC-05
TC-19J	owner_home: decisions with the preview, open requests, today's shifts and week totals	NFR-13, UC-06

Appendix B. Pages and role-based access

Table 47. Role-based access

Feature	Owner	Worker

Feature	Owner	Worker
View the week board	O	O
Add, move or delete a shift; mark a shift absent	O	X
Request a substitute for an own shift	X	O
Accept or answer “can't”	X	O
Approve or reject a swap	O	X
Cancel an own request	X	O
Record attendance	X	O (own shift)
Confirm attendance	O	X
Weekly summary and payroll	O	X
Register workers and fixed shifts	O	X
Settings and Demo tools	O	X
Notification inbox	O	O
See a worker's phone number and wage	O	own only (screen level, NFR-11)

Table 48. Pages, use cases and database functions

Page	Role	Use cases	Database functions
Sign-in (demo)	All	UC-13	`demo_accounts`
Home	Worker	UC-05, UC-04 (entry)	`worker_home`, `accept_sub_request`, `decline_sub_request`
Home	Owner	UC-06, UC-07 (confirm), UC-14 (notice)	`owner_home` (which contains the result of `approval_preview` for each decision), `decide_sub_request`, `confirm_attendance`, `mark_notifications_read`
Schedule	Both	UC-03, UC-04 (entry), UC-06 (result)	`week_board`, `shift_detail`, `generate_week`, `add_shift`, `update_shift`, `delete_shift`, `mark_absent`
Swaps and the request flow	Worker	UC-04, UC-05, UC-12	`my_requests`, `requestable_shifts`, `preview_candidates`, `create_sub_request`, `cancel_sub_request`

Page	Role	Use cases	Database functions
Me	Worker	UC-07 (recording)	`worker_detail`, `attendance_list`, `record_attendance`
Staff	Owner	UC-01	`list_workers`, `worker_detail`, `save_worker`, `save_fixed_schedule`
Pay (Weekly, Monthly)	Owner	UC-08, UC-09	`week_summary`, `payroll_months`, `payroll_month`, `generate_payroll`, `acknowledge_min_wage`, `confirm_payroll`
Settings and Demo tools	Owner	UC-10, UC-13	`get_settings`, `update_settings`, `set_demo_clock`, `reset_demo_data`
Notifications	Both	FR-21	`notifications`, `mark_notifications_read`
(every page)	System Clock	UC-11, UC-14	`run_system_clock`

References

1. Ministry of Employment and Labor, “2026 minimum wage KRW 10,320 per hour,” MOEL notice, Aug. 2025. https://www.moel.go.kr/news/enews/report/enewsView.do?news_seq=18144
2. Korea Policy Briefing, “2026년 시간당 최저임금 10,320원.” https://www.korea.kr/news/policyNewsView.do?newsId=148956812
3. Labor Standards Act (근로기준법), Art. 11, 18, 55, 56; Enforcement Decree Art. 30. https://www.law.go.kr/법령/근로기준법
4. Easy-Law (찾기쉬운 생활법령정보), “기간제 및 단시간근로자: 「근로기준법」 적용의 특례” (paid holidays of part-time workers and the 15-hour threshold). https://www.easylaw.go.kr/CSP/CnpClsMain.laf?csmSeq=1347&ccfNo=3&cciNo=2&cnpClsNo=1
5. Minimum Wage Act (최저임금법), Art. 5 ②; Enforcement Decree Art. 3. https://www.law.go.kr/법령/최저임금법
6. D. A. Gustafson, *Schaum's Outline of Software Engineering*, McGraw-Hill, 2002.
7. I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.
8. OMG, *Unified Modeling Language Specification*, v2.5.1, 2017.
9. PostgreSQL Global Development Group, “Transaction Isolation,” PostgreSQL Documentation (Read Committed: a concurrent UPDATE re-evaluates its WHERE condition on the updated row). https://www.postgresql.org/docs/current/transaction-iso.html
10. Deputy Help Center, “Allow team members to swap or offer shifts.” https://help.deputy.com/hc/en-au/articles/4688726501135-Allow-team-members-to-swap-or-offer-shifts
11. When I Work Help Center, “Getting your shifts covered.” https://help.wheniwork.com/articles/getting-your-shifts-covered/
12. When I Work Help Center, “Giving your team schedule flexibility.” https://help.wheniwork.com/articles/giving-your-team-schedule-flexibility/

13. Homebase Support, help article on trading and covering shifts. https://support.joinhomebase.com/s/article/Trading-and-covering-shifts
14. Deputy Help Center, “Manage different languages in the workplace with Deputy.” https://help.deputy.com/hc/en-au/articles/4621854798223-Manage-different-languages-in-the-workplace-with-Deputy
15. Shiftee, “시프티 200% 활용하기: 근무일정 수정 요청과 전자결재,” Shiftee blog, 2019. https://shiftee.io/ko/blog/article/tipScheduleRequest
16. Shiftee Help Center, frequently asked question on the conditions under which the weekly holiday allowance is recognized. https://shiftee.io/ko/help
17. Shopl, “아르바이트 직원마다 다른 근무 스케줄을 관리하는 방법,” Shopl blog. https://www.shoplworks.com/blog-guides/managing-different-work-schedules-part-time-employees
18. Albami (알바미), service website. https://albame.co.kr/
19. Gig for owners (기그 사장님), app listings on Google Play and the App Store. https://play.google.com/store/apps/details?id=connectfit.logout.ceo; https://apps.apple.com/kr/app/id1523613072
20. Act on the Protection, etc. of Fixed-term and Part-time Employees (기간제 및 단시간근로자 보호 등에 관한 법률), Art. 3 ②, Art. 6; Enforcement Decree Art. 2 and Annex 1. https://www.law.go.kr/법령/기간제및단시간근로자보호등에관한법률
21. Supabase Docs, “Free project pausing.” https://supabase.com/docs/guides/platform/free-project-pausing
22. GitHub Docs, “Events that trigger workflows” (schedule). https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
23. Supabase Docs, “Database Functions” and “Row Level Security.” https://supabase.com/docs/guides/database/functions ; https://supabase.com/docs/guides/database/postgres/row-level-security
24. Lovable Documentation, “Lovable Cloud,” “Database,” “Build with URL” and “Credits and usage.” https://docs.lovable.dev/features/cloud ; https://docs.lovable.dev/features/database ; https://docs.lovable.dev/integrations/build-with-url ; https://docs.lovable.dev/introduction/credits-and-usage

Web pages [1]–[5] and [10]–[22] were accessed on 29 September 2026; [9], [23] and [24] on 10 October 2026.